Robotics and Autonomous Systems 200 (2026) 105402

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

Fault tolerant decentralized collaboration for simultaneous localization and prior map update with stable 2D features☆

Maxime Escourrou *, Joelle Al Hage, Philippe Bonnifait

Université de Technologie de Compiègne, CNRS, Heudiasyc, Compiègne, France

ARTICLE INFO

Keywords:
Collaborative SLAM
Prior map update
Fault detection isolation and recovery
Schmidt–Kalman filter
Decentralized architecture
Multi-robot system

ABSTRACT

Accurate and reliable localization is crucial for safe navigation of autonomous robots. In environments where GNSS performs poorly, robots can rely on stable 2D features stored on prior vector maps, such as OpenStreetMap, which are detected by on-board sensors. This paper proposes a fault tolerant decentralized collaborative method for online localization and map update. We focus on the case of indirect collaboration, where robots only collaborate through the observation of common landmarks. The method relies on the Schmidt–Kalman filter, which handles correlations between robots while minimizing the number of communications. A Kullback–Leibler Average data fusion is also employed. To ensure the integrity of the state estimation, a fault detection, isolation, and recovery approach is proposed, enabling the detection of sensor faults and the correction of map faults through collaboration. The method is evaluated using both simulation and experimental data. Results show that collaboration improves accuracy and consistency of localization and mapping, and improves the ability to detect faults.

1. Introduction

Reliable and accurate localization is crucial for the safe navigation of mobile robots. In environments where GNSS (Global Navigation Satellite System) positioning is unavailable or has degraded performance, robots can rely on stable and geo-referenced features detected using on-board sensors, such as cameras and LiDAR [1,2]. High Definition (HD) maps can be used, and they are versatile since they are sensor-independent. However, they are not always available as they require prior exploration of the area using high-quality equipment. Open data maps, such as OpenStreetMap (OSM), can also be used for localization in outdoor environments [3]. This kind of map contains inaccurately mapped features that must be corrected and updated [4].

Collaboration between robots has shown its advantages for accurate localization and map estimation, relying on the exploitation of redundant information and multiple points of view [5,6].

In this paper, we focus on collaboration between robots via indirect measurements that occur when they observe the same landmark (e.g., building facade). This approach is complementary to direct relative observations, where robots perceive each other directly. However, to simplify the study, such methods will not be considered in this work. Indirect collaboration has the advantage of increasing the range of collaboration and being effective in terms of communication. The approach does not rely on GNSS in order to evaluate its intrinsic performance.

Collaboration is performed through a decentralized architecture to eliminate dependencies on a central unit and to make the system scalable. The Schmidt Kalman Filter (SKF) is a powerful method for such a data fusion architecture since it allows only a part of the state vector to be estimated (collaborating robots), while accounting for the impact of the non-estimated part (non collaborating robots) on the covariance matrix [7].

When two robots collaborate by observing the same landmark, they exchange their poses, covariances, cross-covariances, maps and their observations in the form of relative pose (Fig. 1). A Kullback Leibler Average (KLA) [8] is used to fuse the estimation of the different robots to maintain a global consistency (in the sense that the estimated uncertainty accurately reflects the true error) before applying the SKF. The collaborating robots can then update their poses and maps using the SKF.

A well-known problem when dealing with multi-sensor data fusion for robotics is that measurements can be faulty, leading to inaccurate and inconsistent state estimation that affects the safety of the autonomous agents and their ability to accomplish their mission. These faulty measurements must therefore be detected, and their effects must

☆ This work has been co-financed by the Hauts-de-France Region and Labex MS2T. This work is part of the ANR ToICar project (ANR-21-CE33-0012) and of SIVALab, a joint laboratory between Renault and Heudiasyc.

* Corresponding author.
E-mail addresses: maxime.escourrou@laas.fr (M. Escourrou), joelle.al-hage@hds.utc.fr (J. Al Hage), philippe.bonnifait@hds.utc.fr (P. Bonnifait).

https://doi.org/10.1016/j.robot.2026.105402
Received 26 February 2025; Received in revised form 7 January 2026; Accepted 22 February 2026
Available online 28 February 2026
0921-8890/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

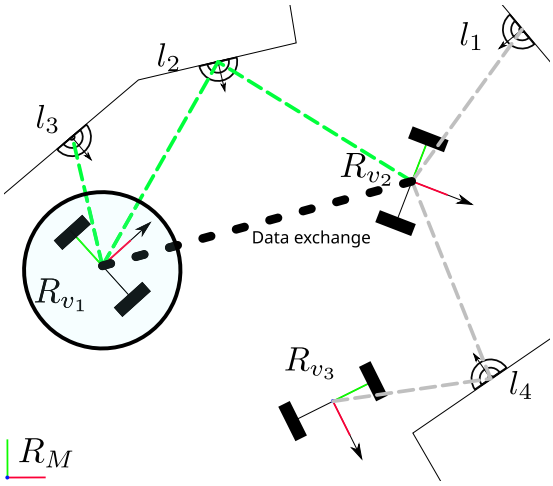


Fig. 1. v_1 collaborates with v_2 via the landmark l_2 and exchanges data with it. In green, observations used by v_1.

be mitigated by adding a fault detection and isolation stage. This stage also addresses map faults (e.g. when a landmark is incorrectly referenced on the prior map) by taking advantage of the collaboration between the robots. The map faults are then isolated and can be corrected.

The contributions of this paper are as follows. (i) We propose a decentralized collaborative localization method with prior map update using robot-landmark observations based on SKF and KLA. (ii) We introduce a novel decentralized collaborative Fault Detection, Isolation, and Recovery (FDIR) method to mitigate sensor and map faults. (iii) We report simulation and experimental results to show the contribution of collaboration and to validate the proposed fault-tolerant approach.

This paper is organized as follows. After a state of the art in Section 2, Section 3 presents the problem statement and the proposed approach. The proposed decentralized collaborative localization and map update method is described in Section 4. Section 5 details the proposed FDIR approach. Simulation results are then reported and discussed in Section 6, and experimental results with observation generations in Section 7.

2. State of the art

The work presented in this paper is related to collaborative Simultaneous Localization and Mapping (SLAM) with prior map, while dealing with faulty data originating from sensor measurements or the prior map.

2.1. Simultaneous localization and mapping

SLAM is defined as the simultaneous estimation of the robot's state and its environment, while processing data sequentially [9]. SLAM systems are commonly divided into two parts, each being the subject of a different field of study: the front-end and the back-end [5]. The front-end is the part in charge of processing sensor data, in order to extract observations. The back-end produces the final state estimate using the observations provided by the front-end. This relates to research in optimization and filtering.

Optimization-based approaches, known also as full SLAM, aim to optimize the complete trajectory of the robot considering all current and past measurements, through objective functions. They can be divided into bundle adjustment (BA) and pose-graph [10,11].

Filter-based approaches, also known as online SLAM, estimate the state using recursive Bayesian filters such as the Extended Kalman Filter (EKF) [12,13], and the Extended Information Filter (EIF) [14, 15]. SLAM based on Kalman filter is not new and suffers from inconsistency problems that originate from linearization and heading uncertainty [16].

Some authors like in [17] compared the two classes of methods using monocular cameras. The complexity of filtering approaches increases with the square of the map size. Some filtering methods can deal with this problem. For example, the sparse EIF adds a sparsification step where the matrices become almost block-diagonal [18]. The Schmidt–Kalman Filter estimates only a part of the state, which reduces the size of the matrices to manipulate [7,19]. Other studies have shown that SLAM with invariant EKF behaves similarly to optimization methods in terms of accuracy and consistency [20,21]. There are also techniques to reduce the computational time of filtering approach [14].

The choice of optimization-based or filter-based method lies on multiple criteria and on the targeted application. In the context of a system that needs to estimate its uncertainty (e.g. using an estimate of the covariance), filtering methods are particularly suitable and allow for a straightforward implementation of a fault detection step. Optimization methods that account for uncertainty require additional operations, which are sensitive to convergence issues (local minima) and can lead to overconfidence [22].

2.2. Collaboration for localization and mapping

Collaboration between robots enhances SLAM methods by improving and robustifying the state estimation through the sharing of relevant information. This requires the choice of a data fusion architecture. Although the centralized architecture is easy to implement, it relies on a central fusion unit and assumes unlimited computational resources. In contrast, a decentralized architecture removes the dependency on a central processing unit and distributes the computation among the robots. This architecture is more fault-tolerant, as it removes the single point of failure represented by the central unit. However, it can suffer from the reuse of the same information multiple times while treating it as uncorrelated. This phenomenon is known as data incest leading to overconfidence in the state estimation [23].

Collaborative Localization (CL) has been addressed in several works to enhance localization by taking advantage of inter-robot observations [24–27]. Typically, robots must be within each other's field of view, limiting the collaboration range.

Collaborative SLAM (C-SLAM) allows to maintain a representation of the environment. In [28], the authors present the collaborative sparse EIF where each robot operates independently and builds its own map. Collaboration only takes place through the fusion of robot's map. This approach requires a permanent communication between robots. In [14], a decentralized C-SLAM is proposed, based on an uncertain prior map. The robots use an EIF to represent the state of the robots and the map. To avoid data incest, robot communications are synchronized. Another decentralized architecture based on BA, EKF and covariance intersection was proposed in [6] to localize a fleet of micro aerial vehicles while building and refining the map. Robots collaborate by exchanging their maps and estimating their relative pose through the match of common features. This is a collaborative approach based on indirect observations between robots. However, this method does not take into account map uncertainty. iMESA [29] uses an incremental distributed optimization approach that allows robots to share variables and local estimates while integrating new measurements and consensus constraints. Although this enables large-scale, long-term multi-robot operations, the method remains vulnerable to outliers and communication dropouts, reducing its overall robustness. Moreover, approaches based on neural networks can be found in the literature, like the one proposed in [30].

The presented C-SLAM methods are generally highly dependent on communications. Likewise, computational complexity must be considered when dealing with robots with limited computational capabilities.

2.3. Fault tolerant data fusion

Faulty sensor measurements can directly impact the integrity and accuracy of the estimation of robot poses and landmarks. Furthermore, in collaborative systems, faults in a robot's measurements can propagate throughout the system, potentially affecting the mission. The system must detect and mitigate these faults by relying on diagnosis methods [31–33].

In the literature, a distinction is made between diagnosis methods based on hardware redundancy and analytical redundancy [31]. Analytical redundancy compares the actual process with a model. It can be divided into model-based (such as Kalman filter), data-based and knowledge-based methods [31,34].

Fault detection and isolation based on a bank of EKF for SLAM approach was proposed in [35], where two lidars and three odometry methods (gyroscope, encoders and indoor GNSS) are used in different combinations, in a redundant manner. It is thus possible to isolate a fault coming from one of the sensors or one of the filters using a signature matrix [33]. In [36], an interactive multiple model Kalman filter was proposed where faults can be mitigated using redundant sensors. In [37], the authors proposed a fault tolerant localization method based on redundant estimation system. Various residuals are computed based on different sensor combinations, while assuming only one fault at a time. These methods need to run multiple filters simultaneously, which can consume many computational resources. In [38], a fault tolerant multi-sensor data fusion for UAV (Unmanned Aerial Vehicle) based on voting system was proposed.

In multi-robot systems, a distinction can be made between methods that detect local faults at the level of each robot, known as endogenous fault detection [35], and those that detect faults originating from other robots, known as exogenous fault detection [39]. A fault originating from one robot can thus be detected by another and shared with the rest, thereby enhancing the robustness of the overall system. While some studies also address the detection of faulty robots behaviors [40,41], this paper focuses on sensor and map faults.

The authors in [42] proposed a distributed Fault Detection and Exclusion (FDE) for a multi-robot system. The approach is based on the computation of the informational contributions of each observation and on residuals calculated using the Kullback–Leibler Divergence. A data-driven approach is used for threshold setting. In [43], a data-driven approach was used for fault diagnosis in a decentralized cooperative system. The approach is based on pre-trained detection decision tree for fault classification and a random forest for fault isolation. In [44], a layered fault detection and isolation stage was proposed where faults are categorized into two groups: those that can be monitored on a single robot, and those that can be detected using collaborative techniques.

3. Problem statement and proposed approach

Consider N mobile robots that navigate in a 2D environment with M landmarks. Each robot is similarly equipped with a lidar sensor, wheel odometry, and a wireless communication system.

At the start of their missions, the robots retrieve an initial prior map (e.g. OpenStreetMap), which is the same for all and assumed to be globally accurate but contains some errors and faults. Each landmark is represented by a unique identifier (ID), retrieved from the prior map, and by its pose $X_l = \begin{bmatrix} x & y & \theta \end{bmatrix}_l^T$. The landmarks are initially referenced with some uncertainty in the prior map and will be refined over time as the robots detect them. In our experimental setup, the landmarks represent building facades. The robots are represented as 2D poses, denoted $X_v = \begin{bmatrix} x & y & \theta \end{bmatrix}_v^T$. Both landmarks' and robots' poses are expressed in the map frame R_M (Fig. 1). The set of robots is denoted V with elements v_i, $i = \{1, \dots, N\}$. The set of landmarks is denoted L with elements l_j, $j = \{1, \dots, M\}$. The robots collaborating with the ego-robot v_i will be grouped into a set V_i, with $v_i \in V_i$.

The collaboration between robots only takes place when they observe common landmarks (e.g. building facades) at the same time. This strategy has the advantage of increasing the range of collaboration without requiring a direct line of sight between the robots. Information is then exchanged only between collaborating robots in order to improve their localization and maps. The limitations of communication, including energy cost, latency, and availability, motivate the adoption of a decentralized architecture that minimizes the number of communications. It is assumed that robot-to-robot communication is bidirectional.

When features are detected, each robot locally broadcasts within a given radius the IDs of the observed landmarks using wireless communication. These messages are very short, consuming minimal bandwidth. Other robots in the vicinity check if the received IDs correspond to any of their own observations. If there is at least one match, they engage in peer-to-peer communication with the sender, sending back all the necessary information required for the decentralized estimation process without requiring any handshake or acknowledgment. This includes sharing their state (poses and map) with the associated covariance matrix in addition to the observations. The other robots carry out the same procedure. As communication delays are unavoidable, all observations are time-stamped with a common date and each robot maintains a buffer to handle out-of-sequence data to apply updates at the right times [45].

The main goal is to achieve a consistent collaborative localization and map updates, by detecting, isolating, and mitigating faults in sensor measurements and the prior map. In this study, we distinguish three types of faults (Fig. 2): observation fault, landmark fault, and odometry fault, with the assumption that no more than two faults occur simultaneously.

The core of the approach is based on the use of the SKF, which is well suited to a decentralized architecture with communication constraints [25]. This approach is also robust to message loss, enabling the robots to continue their consistent state estimation using only their own observations. The SKF allows estimating only the state of collaborating robots while accounting for the influence of non-collaborating robots through the covariance matrix. To maintain consistent estimates, a data fusion using KLA is applied prior to each collaborative update step.

The general architecture of the method is presented in Fig. 3. The lidar point cloud of each robot is processed to generate a list of observations, which are then augmented with collaborative observations from robots observing common landmarks. The SKF is applied at the level of each robot.

A decentralized Fault Detection, Isolation, and Recovery (FDIR) process is proposed based on SKF models. Each robot is able to exclude erroneous measurements generated by its perception sensor or received from other robots. Additionally, landmark errors can be detected and corrected by taking advantage of the redundancy in the multi-robot system (Fig. 3). In the case where more than two faults occur at the same time, the landmark faults cannot be isolated. In this case, the proposed approach excludes observations whose residuals exceed predefined thresholds, ensuring the availability of an accurate solution.

4. Decentralized collaborative localization and map update

4.1. State estimation based on SKF

Each robot has its own filter that can operate in a standalone mode (without collaboration). The robot's observation vector is the concatenation of its local observations and the ones of the common landmark(s) made by the other robots.

The joint state vector is composed of the poses of all robots and landmarks:

$$X = \begin{bmatrix} (X_v)_{v \in V} \\ (X_l)_{l \in L} \end{bmatrix}. \tag{1}$$

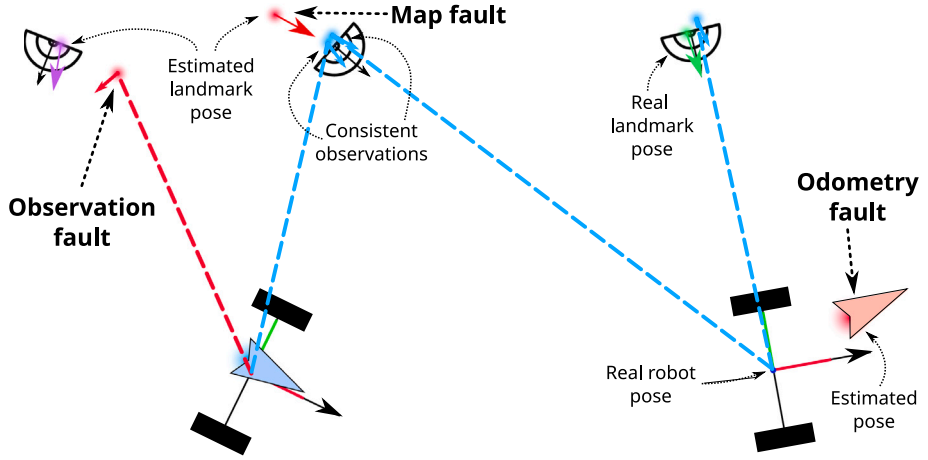


Fig. 2. Illustration of the types of faults considered in this paper.

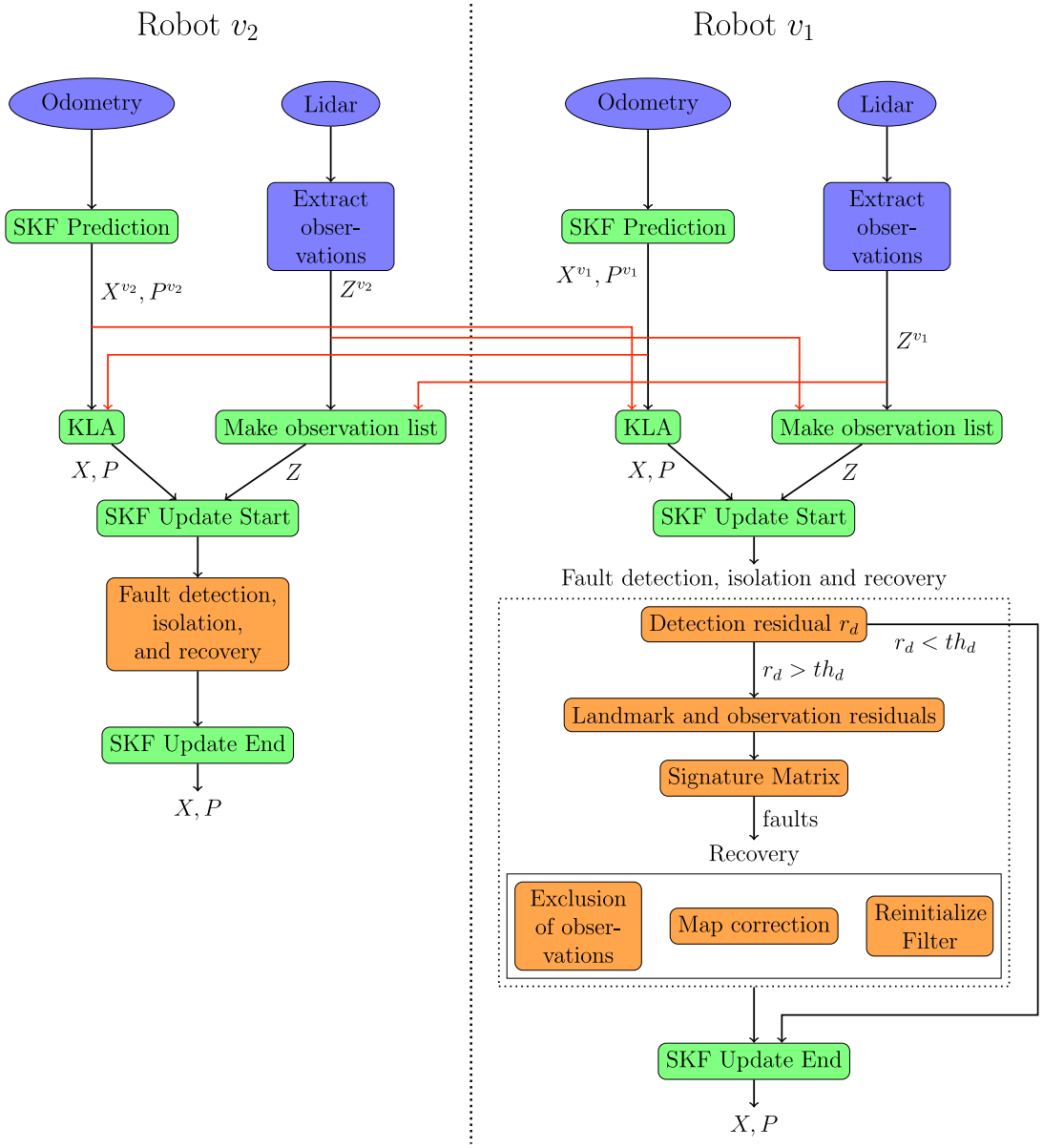


Fig. 3. Illustration of the decentralized architecture. Here, robot v_1 is collaborating with robot v_2. X^{v_1} and P^{v_1} (respectively X^{v_2} and P^{v_2}) are the full states (robot poses and map) estimated by v_1 (respectively v_2).

One of the main objectives of the proposed approach is to limit the number of communications. The SKF is well suited to this problem since it allows estimating only a part of the state [7]. It divides the state vector into two parts. A first part is estimated, denoted s, and a second part is not estimated but its impact on the covariance is determined. It is the *parameters part* denoted p. The state vector in Eq. (1) is then reorganized as:

$$X = \begin{bmatrix} X_s \\ X_p \end{bmatrix}. \tag{2}$$

The s part is composed of the map, the robot v_i and the robots collaborating with v_i:

$$X_s = \begin{bmatrix} (X_v)_{v \in V_i} \\ (X_l)_{l \in L} \end{bmatrix}. \tag{3}$$

The p part contains the robots not collaborating with v_i:

$$X_p = \left[X_v\right]_{v \notin V_i} = \left[X_v\right]_{v \in \bar{V}_i}. \tag{4}$$

Likewise, the covariance matrix can be reorganized into

$$P = \begin{bmatrix} P_{ss} & P_{sp} \\ P_{ps} & P_{pp} \end{bmatrix}. \tag{5}$$

SKF allows a different division of the state at each step, while keeping consistency. We have chosen to keep all the landmarks in the estimated part for implementation simplification.

4.1.1. SKF prediction step

The prediction step of the SKF can be done in a similar manner to a classical EKF by applying the evolution model only on the s part. The p part is kept static.

During the prediction step, the s part contains the ego-robot v_i and the map. The p part contains the other robots.

The prediction step based on the SKF is given by:

$$\begin{pmatrix} X_s \\ X_p \end{pmatrix}_{k+1|k} = \begin{pmatrix} f\left(X_{s,k|k}, u_k\right) \\ X_{p,k|k} \end{pmatrix}. \tag{6}$$

The evolution model uses only the input vector corresponding to robot v_i. It contains the linear v and angular ω velocity measurements $u_{i,k} = \begin{bmatrix} v & w \end{bmatrix}^T_{i,k}$. The odometry data of the other robots being not accessible, the prediction is then limited to robot v_i.

The predicted covariance matrix of the ego-robot v_i is

$$P_{v_i,k+1|k} = F_{v_i,k} P_{v_i,k|k} F^T_{v_i,k} + G_{v_i,k} Q_{u_i} G^T_{v_i,k} + Q_i, \tag{7}$$

with $F_{v_i,k}$ and $G_{v_i,k}$ the Jacobian matrices of the evolution model, Q_{u_i} the covariance of the input noise, and Q_i the one of the state noise.

Since the landmarks are not moving, their prediction is static and no uncertainty is added.

The poses of the other robots (in the p part) are not estimated. A covariance Q_p is added to account for their possible motion:

$$X_{v_j,k+1|k} = X_{v_j,k|k}, \tag{8}$$

$$P_{v_j,k+1|k} = P_{v_j,k|k} + Q_p. \tag{9}$$

The predicted cross-covariances between v_i and an element j, being either a robot or a landmark, is given by:

$$P_{v_i j,k+1|k} = F_{v_i,k} P_{v_i j,k|k}. \tag{10}$$

4.1.2. SKF update step

For the update step, we consider the case of the ego-robot v_i and the set of robots V_i observing at least one landmark in common with v_i.

The observation vector Z_k of v_i is limited to its own observations and the observations of the common landmarks made by the other robots (Fig. 1).

The observation vector contains the landmark poses in the observing robot frame. For example, the observation model of landmark l_j by robot v_i is

$$Z^{v_i}_{l_j} = h^{v_i}_{l_j}(X) + \beta^{v_i}_{l_j} \tag{11}$$

$$= \mathcal{R}^T_{v_i}\left(X_{l_j} - X_{v_i}\right) + \beta^{v_i}_{l_j}, \tag{12}$$

where

$$\mathcal{R}^T_{v_i} = \begin{pmatrix} \cos\theta_{v_i} & \sin\theta_{v_i} & 0 \\ -\sin\theta_{v_i} & \cos\theta_{v_i} & 0 \\ 0 & 0 & 1 \end{pmatrix}, \tag{13}$$

and $\beta^{v_i}_{l_j}$ is the measurement noise considered as white Gaussian with covariance $R^{v_i}_{l_j}$. Please note that the orientation difference is kept between $-\pi$ and π.

H_k is the Jacobian matrix of the observation model $h(X)$:

$$H = \begin{pmatrix} H_v & H_l \end{pmatrix}, \tag{14}$$

where H_v represents the robots part

$$H_v = \begin{pmatrix} \left(H_v\right)^{v_i}_{l_j} \\ \vdots \end{pmatrix}, \tag{15}$$

and H_l represents the landmarks part

$$H_l = \begin{pmatrix} \left(H_l\right)^{v_i}_{l_j} \\ \vdots \end{pmatrix}. \tag{16}$$

Each block $\left(H_v\right)^{v_i}_{l_j}$ is defined by

$$\left(H_v\right)^{v_i}_{l_j} = \begin{cases} \mathcal{R}^T_{v_i} \Gamma^{v_i}_{l_j} & \text{if column block index is } i \\ \mathbb{0}_{3\times 3} & \text{otherwise,} \end{cases} \tag{17}$$

with

$$\Gamma^{v_i}_{l_j} = \begin{pmatrix} -1 & 0 & y_{l_j} - y_{v_i} \\ 0 & -1 & -\left(x_{l_j} - x_{v_i}\right) \\ 0 & 0 & -1 \end{pmatrix}, \tag{18}$$

and each landmark block $\left(H_l\right)^{v_i}_{l_j}$ is defined by

$$\left(H_l\right)^{v_i}_{l_j} = \begin{cases} \mathcal{R}^T_{v_i} & \text{if column block index is } j \\ \mathbb{0}_{3\times 3} & \text{otherwise.} \end{cases} \tag{19}$$

For SKF, H is reorganized into s and p parts:

$$H = \begin{pmatrix} H_s & H_p \end{pmatrix}. \tag{20}$$

To better illustrate the shape of H_s and H_p, let us take the example of three robots and two landmarks. We consider v_1 observing l_1 and l_2, and v_2 observing l_2. No observation is made by v_3. Robots v_1 and v_2 collaborate through the observation of l_2. From the point of view of v_1, the matrix H is:

$$H = \begin{array}{c} \begin{matrix} v_1 & v_2 & l_1 & l_2 & v_3 \end{matrix} \\ \begin{pmatrix} \mathcal{R}^T_{v_1} \Gamma^{l_1}_{v_1} & \mathbb{0} & \mathcal{R}^T_{v_1} & \mathbb{0} & \mathbb{0} \\ \mathcal{R}^T_{v_1} \Gamma^{l_2}_{v_1} & \mathbb{0} & \mathbb{0} & \mathcal{R}^T_{v_1} & \mathbb{0} \\ \mathbb{0} & \mathcal{R}^T_{v_2} \Gamma^{l_2}_{v_2} & \mathbb{0} & \mathcal{R}^T_{v_2} & \mathbb{0} \end{pmatrix} \end{array} \begin{matrix} Z^{v_1}_{l_1} \\ Z^{v_1}_{l_2} \\ Z^{v_2}_{l_2} \end{matrix}. \tag{21}$$

The s part contains v_1, v_2 and the landmarks. The p part corresponds to v_3 represented by the last column containing only $\mathbb{0}$. With our choice to not take into account the observations of non-collaborating robots, we will always have $H_p = \mathbb{0}$.

The Kalman gain of the SKF can be decomposed into K_s and K_p. With the SKF, the p part is not updated, this can be done by setting $K_p = \mathbb{0}$. This choice is possible because P_{ss} and P_{sp} are independent of K_p [7].

By taking advantage of $H_p = \mathbb{0}$, the updated state vector and covariance matrices of the SKF are simplified as follows:

$$X_{s,k|k} = X_{s,k|k-1} + K_{s,k}\left(Z_k - h\left(X_{s,k|k-1}\right)\right), \tag{22}$$

$$X_{p,k|k} = X_{p,k|k-1}, \tag{23}$$

$$P_{ss,k|k} = \left(I - K_{s,k}H_{s,k}\right)P_{ss,k|k-1}, \tag{24}$$

$$P_{sp,k|k} = \left(I - K_{s,k}H_{s,k}\right)P_{sp,k|k-1}, \tag{25}$$

$$P_{pp,k|k} = P_{pp,k|k-1}. \tag{26}$$

The Kalman gain K_s can be written as:

$$K_{s,k} = P_{ss,k|k-1}H_{s,k}^T S_k^{-1}, \tag{27}$$

with:

$$S_k = H_{s,k}P_{ss,k|k-1}H_{s,k}^T + R_k. \tag{28}$$

R_k is the observation noise covariance matrix supposed to be block-diagonal with elements $R_{l_j}^{v_i}$.

4.2. Fusion of multiple state estimates using KLA

The proposed decentralized architecture with communication constraints leads to a multiplication of map estimates, and several versions of robot-map cross-covariances. Indeed, each robot estimates its own map, and during update steps, it corrects it collaboratively with the observations of the other robots. This leads to differences in map estimates as well as in the cross-covariance with the robots.

In the following, we denote by an exponent the robot possessing the variable. For example, the full state (all robot poses and the map) estimated by robot v_i is denoted $X^{v_i} = \begin{pmatrix} X_V^{v_i} \\ X_L^{v_i} \end{pmatrix}$. The associated covariance is denoted P^{v_i}.

In order to fuse the information from the different robots in collaboration, the KLA is used before each SKF update step.

For Gaussian distributions, the KLA simplifies to the average of information vectors and information matrices [8]:

$$\bar{P}_{k|k-1}^{-1} = \frac{1}{|V_i|}\sum_{v_j \in V_i}\left(P_{k|k-1}^{v_j}\right)^{-1}, \tag{29}$$

$$\bar{X}_{k|k-1} = \bar{P}_{k|k-1}\frac{1}{|V_i|}\sum_{v_j \in V_i}\left(P_{k|k-1}^{v_j}\right)^{-1}X_{k|k-1}^{v_j}. \tag{30}$$

with $|V_i|$ the number of robots in collaboration. It should be noted that a fusion using the covariance intersection is also possible, but not desirable. Indeed, it tends to select one or the other of the estimates [46]. Likewise, the optimization problem to be solved with the CI can be costly in the case of merging n sources. The KLA corresponds to a CI where the weights are fairly distributed.

After the KLA step, the SKF update is applied using $\bar{X}_{k|k-1}$ and $\bar{P}_{k|k-1}$ as prediction.

Performing a data fusion with the KLA on the full state requires the estimates to be consistent. The goal is then to have consistent estimates of the other robots without using their odometry data. For this reason, the model uncertainty Q_p is increased for the other robots using Eq. (9). In this way, the estimate of $X_{v_j,k+1|k}$ by v_i remains consistent, although probably erroneous, and the KLA will correct it.

4.3. Communication bandwidth and computational load

The system requires data exchange between robots. When a robot observes a landmark, it only broadcasts the ID of the landmark, in a given radius, representing a short message (several bytes of payload). Once collaboration is established, it sends all the necessary information. This "heavy" sharing is limited to peer-to-peer communications between robots observing the same landmark.

The bandwidth usage of our method is $O\left(N^2 + M^2\right)$, where N and M are the number of robots and landmarks, respectively. This is lower than the $O(N^3 + 2MN^2 + M^2N)$ complexity of the *Channel Filter* [47], a well known decentralized method. For this result, we make the assumption that the number of neighbors is small compared to the total number of robots. The detailed calculations and results can be found in Appendix. Some optimizations can be done in order to improve bandwidth performance. For example, if KLA is only computed at strategic moments, we would not need to exchange the full state, which reduces the bandwidth usage. Indeed, when robots are continuously collaborating, they share almost the same information, so it is not necessary to perform the KLA at every step. For example, the KLA could be executed only when two robots begin collaborating or at a low frequency during ongoing collaboration.

Let us now consider the complexity corresponding to the computation performed by a single robot.

From a computational complexity perspective, SKF outperforms EKF when the size of the s part is small relative to the full state size $n = s+p$. Assuming that the number of observations is small compared to the map size, the resulting complexity is $O\left(ns^2\right)$, whereas the standard EKF exhibits a complexity of $O\left(n^3\right)$. While the KLA brings the complexity to $O\left(n^3\right)$ due to matrix inversion, optimization can be applied. For example, the KLA can be computed only at the strategic moments mentioned above.

In conclusion, the proposed method scales very well with the number of robots compared to other decentralized methods, particularly in terms of bandwidth usage (second order in the state size). Likewise, the computational complexity for SKF scales efficiently with the total number of robots since the s part contains only the subset of collaborating robots. The more robots, the more advantageous the proposed method becomes. Moreover, it does not require a continuously connected network, and message losses can be tolerated without much affecting the overall system performance.

5. Fault detection, isolation, and recovery

Fault tolerance is the aptitude of a system to maintain operation in the presence of faults. In this paper, three fault types are considered (Fig. 2):

- An *observation fault* occurs when the sensor fails to accurately detect a landmark due to environmental noise or errors in the data processing, resulting in a measurement that deviates from the true value. Misassociation issues will be detected as observation faults. It should be noted that misassociation faults are not frequent with our chosen features, as the observed facades are far enough apart to cause such problems.
- A *map landmark fault* occurs when a landmark is incorrectly referenced on the map. These are landmarks far from their real poses with non-representative covariance. Missing landmarks or new landmarks are not considered in this study.
- An *odometry fault* occurs when the estimation process diverges due to an odometry issue or a previous undetected fault.

These faults have a direct effect on the accuracy and integrity of state estimation. An FDIR step is then included to overcome this problem. The FDIR is done through a decentralized architecture at each robot level. If a fault is detected, an isolation and a recovery step is performed based on the fault type to mitigate its effect. The proposed method uses residuals which are constructed in two spaces: the observation space and the state space. The advantage of the collaboration will be highlighted for the FDIR step.

Note that some time indices, k, are omitted to simplify the readability of the equations.

Table 1
Signature matrix without collaboration.

Residuals \ Faults	Map Landmarks		Observations of v_1		Odometry
	l_1	l_2	$Z_{l_1}^{v_1}$	$Z_{l_2}^{v_1}$	odo_1
$r_{l_1}^{v_1}$	1	0	1	0	1
$r_{l_2}^{v_1}$	0	1	0	1	1

5.1. Fault detection

The first step of the FDIR is the detection of the presence of faults. For that, a detection residual is used. This residual is the squared Mahalanobis distance between the observation vector Z_k and its prediction $\hat{Z}_k$:

$$r_{d,k} = \left(Z_k - \hat{Z}_k\right)^T S_k^{-1} \left(Z_k - \hat{Z}_k\right), \tag{31}$$

where S_k is the innovation covariance matrix, Eq. (28).

Each robot computes its own detection residual using the observation vector composed of local and collaborative observations, as explained in Section 4. This residual is compared to a threshold obtained from a Chi-squared distribution with a given false alarm probability. If the residual exceeds the threshold, a fault is detected, and the origin should be determined based on a set of residuals.

5.2. Fault isolation

The second step of the FDIR is to determine the origin of the fault to perform the right action. For this reason, a set of residuals in the form of squared Mahalanobis distances between the observations and their predictions are computed. In the case of landmark l_j observed by robot v_i, the residual is:

$$r_{l_j}^{v_i} = \left(Z_{l_j}^{v_i} - \hat{Z}_{l_j}^{v_i}\right)^T \left(S_{l_j}^{v_i}\right)^{-1} \left(Z_{l_j}^{v_i} - \hat{Z}_{l_j}^{v_i}\right), \tag{32}$$

where $\hat{Z}_{l_j}^{v_i}$ is computed using the observation model, Eq. (12), and $S_{l_j}^{v_i}$ is one block-row of Eq. (28). These residuals are the observation residuals.

$r_{l_j}^{v_i}$ is sensitive to an observation fault through $Z_{l_j}^{v_i}$ and to a fault of the landmark state l_j via $X_{l_j,k|k-1}$. It is also sensitive to an odometry fault through $X_{v_i,k|k-1}$.

5.2.1. Single fault

To determine the origin of faults, a signature matrix is used. This matrix links the set of residuals to the potential faults, where each row corresponds to a residual and each column to a type of fault. The sensitivity of a residual to a fault is marked by "1". The matrix can localize a fault if all the columns are different, meaning that each fault is associated to a unique combination of residuals.

Table 1 shows an example of the signature matrix when v_1 observes two landmarks l_1 and l_2. Faults can appear on the observations $Z_{l_1}^{v_1}$ and $Z_{l_2}^{v_1}$, the map landmarks l_1 and l_2 and the odometry. It can be seen that this matrix has identical columns, making fault isolation impossible. For example, if $r_{l_1}^{v_1}$ exceeds the threshold, the fault signature matches with l_1 and $Z_{l_1}^{v_1}$. In order to be able to distinguish between a landmark fault and an observation fault, redundancy is necessary. This is where collaboration between robots is very important, when the same landmark can be observed by several robots.

Table 2 shows the same situation with the addition of another robot v_2 that collaborates with robot v_1 by observing also landmarks l_1 and l_2. The columns corresponding to multiple faults will be discussed in the next paragraph. In this case, a landmark fault l_1 has a different signature from a fault of an observation of l_1 (e.g., $Z_{l_1}^{v_1}$). Therefore, thanks to the collaboration, it is possible to isolate map fault and observation fault, under the assumption of one fault at a time.

5.2.2. Multiple simultaneous faults

Now suppose that two faults can occur at the same time. In this case, the signature matrix is completed by combining the columns through a logical OR operation [33]. This creates new signatures corresponding to the combined faults as shown in Table 2. This matrix cannot differentiate between a landmark fault (e.g., l_1) and two observations faults of this same landmark (e.g., $Z_{l_1}^{v_1}$ and $Z_{l_1}^{v_2}$) since they have the same signature.

To deal with this problem, another type of residual can be added. The proposed residual is designed to be a map-independent quantity and is computed when two robots collaborate by observing the same landmark. It compares multiple pose estimates of the same landmark. Each estimate is computed using one observation at a time. The idea behind is that if two estimates based on different observations are consistent but the observations are both far from their prediction, then the fault comes from the map. It is then possible to distinguish between multiple faults $\left\{Z_{v_1}^{l_1}, Z_{v_2}^{l_1}\right\}$ and landmark fault l_1.

Contrary to the first residual computed in the observation space, the landmark residual is computed in the state space. It is obtained using the squared Mahalanobis distance between both estimates of the same landmark expressed in the map frame:

$$r_{l_j}^{v_i v_k} = \left({}^M Z_{l_j}^{v_i} - {}^M Z_{l_j}^{v_k}\right)^T \left({}^M R_{l_j}^{v_i} + {}^M R_{l_j}^{v_k}\right)^{-1} \left({}^M Z_{l_j}^{v_i} - {}^M Z_{l_j}^{v_k}\right), \tag{33}$$

where ${}^M Z_{l_j}^{v_i}$ and ${}^M Z_{l_j}^{v_k}$ are the poses of the landmark l_j estimated by v_i and v_k in the map frame. ${}^M R_{l_j}^{v_i}$ and ${}^M R_{l_j}^{v_k}$ are their covariances in the map frame. These pose estimates are obtained by applying a transformation to the observations:

$${}^M Z_{l_j}^{v_i} = \mathcal{R}_{v_i,k|k-1} Z_{l_j}^{v_i} + X_{v_i,k|k-1}, \tag{34}$$

with $\mathcal{R}_{v_i,k|k-1}$ the rotation matrix between the robot frame and the map frame, as defined in Eq. (13).

The covariance matrix associated to ${}^M Z_{l_j}^{v_i}$ is given by [48]:

$${}^M R_{l_j}^{v_i} = J \begin{pmatrix} P_{v_i} & \mathbb{0} \\ \mathbb{0} & R_{l_j}^{v_i} \end{pmatrix} J^T, \tag{35}$$

where J is the Jacobian matrix $\frac{\partial\left({}^M Z_{l_j}^{v_i}\right)}{\partial\left(X_{v_i}, Z_{l_j}^{v_i}\right)}$:

$$J = \begin{pmatrix} 1 & 0 & -\left(y_{v_i} - {}^M y_{l_j}^{v_i}\right) & \cos\theta_{v_i} & -\sin\theta_{v_i} & 0 \\ 0 & 1 & \left(x_{v_i} - {}^M x_{l_j}^{v_i}\right) & \sin\theta_{v_i} & \cos\theta_{v_i} & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}. \tag{36}$$

P_{v_i} is the state covariance matrix of the pose of the robot v_i and $R_{l_j}^{v_i}$ is the observation covariance matrix.

The landmark residual $r_{l_j}^{v_i v_k}$ is then sensitive to faults on the two observations ($Z_{l_j}^{v_i}$ and $Z_{l_j}^{v_k}$) and a fault on the odometry of the two robots.

The signature matrix after adding these residuals is given in Table 3. The matrix can now differentiate between two observation faults of the same landmark (e.g., $Z_{l_1}^{v_1}$ and $Z_{l_1}^{v_2}$) and landmark fault (e.g., l_1).

The proposed signature matrix can efficiently perform fault isolation up to two faults under some assumptions:

- A map fault can be isolated if at least two robots observe the same landmark. Collaboration among robots is crucial for effectively isolating map landmark faults.

Table 2
Signature matrix with collaboration. For simplification, the empty case refers to "0".

Residuals \ Faults	Map		Obs. of v_1		Obs. of v_2		Odometry		Multiple		
	l_1	l_2	$Z^{v_1}_{l_1}$	$Z^{v_1}_{l_2}$	$Z^{v_2}_{l_1}$	$Z^{v_2}_{l_2}$	odo_1	odo_2	$\{Z^{v_1}_{l_1}, Z^{v_2}_{l_1}\}$	$\{Z^{v_1}_{l_2}, Z^{v_2}_{l_2}\}$	…
$r^{v_1}_{l_1}$	1		1				1		1		…
$r^{v_1}_{l_2}$		1		1			1			1	…
$r^{v_2}_{l_1}$	1				1			1	1		…
$r^{v_2}_{l_2}$		1				1		1		1	…

Table 3
Signature matrix with the two types of residuals.

Residuals \ Faults	Map		Obs. of v_1		Obs. of v_2		Odometry		Multiple		
	l_1	l_2	$Z^{v_1}_{l_1}$	$Z^{v_1}_{l_2}$	$Z^{v_2}_{l_1}$	$Z^{v_2}_{l_2}$	odo_1	odo_2	$\{Z^{v_1}_{l_1}, Z^{v_2}_{l_1}\}$	$\{Z^{v_1}_{l_2}, Z^{v_2}_{l_2}\}$	…
$r^{v_1}_{l_1}$	1		1				1		1		…
$r^{v_1}_{l_2}$		1		1			1			1	…
$r^{v_2}_{l_1}$	1				1			1	1		…
$r^{v_2}_{l_2}$		1				1		1		1	…
$r^{v_1,v_2}_{l_1}$			1		1		1	1	1		…
$r^{v_1,v_2}_{l_2}$				1		1	1	1		1	…

- The signature of the "odometry" fault is the same as a perception sensor issue on one robot, where all observations of the same robot are affected. To isolate an "odometry" fault (divergence), a level of redundancy is required in the system. This redundancy is obtained when the robot observes at least three landmarks. The underlying idea is that the probability of having three or more faulty observations simultaneously is relatively low. If the robot observes less than three landmarks, the problem is considered to arise from observations and the odometry fault cannot be isolated.
- A simultaneous fault of a landmark and an observation of this landmark cannot be isolated. The observation will be considered faulty.

In the following, we will assume a maximum of two simultaneous faults at a time and consider that all three assumptions are met.

In the case where there are more than two simultaneous faults, or when the signature matrix cannot determine the exact origin of fault due to an unverified assumption, the observations whose residuals exceed their threshold are considered as faulty. In real world, more than two simultaneous faults can occur, often at the observation level. In such cases, our method can exclude all the faulty observations from the data fusion. If faults occur simultaneously on a landmark and on an observation of this landmark, they would produce the same signature as two faulty measurements. However, this scenario is less likely. Although the signature matrix cannot determine the exact origin of the fault, we proceed to the exclusion of observations whose residuals exceed their respective threshold. This means that the map cannot be corrected at that time using the FDIR strategy, which makes sense in the absence of high-quality measurements for the landmark.

5.3. Fault recovery

After isolating the origin of the fault, the right action must be taken so faults do not affect the accuracy and integrity of the state estimates. Different actions are carried out depending on the type of fault, namely, the exclusion of erroneous observations or the correction of the landmark pose. This step is done also through a decentralized architecture at each robot level.

Fault exclusion. In the case of an observation fault, a simple action consists in the exclusion of the faulty observations from the data fusion. Each robot may exclude different observations.

Map correction. In the case of a landmark fault, the exclusion cannot be done since the fault comes from the state itself. In this case, the contribution of the observations is increased with respect to the estimation of the erroneous landmark by increasing its covariance:

$$P^{+}_{l_i,k|k-1} = P_{l_i,k|k-1} + A, \tag{37}$$

where A is a positive-defined matrix. The volume of this matrix is defined as a function of the distance d between the mean of the m observations of the landmark, $\bar{Z}_{l_i}$, and the predicted state of the landmark, $X_{l_i,k|k-1}$:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.5 \end{pmatrix} d^2\eta, \tag{38}$$

$$d = \|X_{l_i,k|k-1} - \bar{Z}_{l_i}\| \quad \text{using only } x \text{ and } y, \tag{39}$$

$$\bar{Z}_{l_i} = \frac{1}{m}\sum_j {}^{M}Z^{v_j}_{l_i}, \tag{40}$$

where η is a size factor, arbitrarily set to 9, and ${}^{M}Z^{v_j}_{l_i}$ is defined in Eq. (34).

Thanks to this increase in uncertainty of the landmark pose, its estimated pose will be corrected based on the available observations in the following update step.

Filter reset. In the case of odometry fault, a signal must be sent to reinitialize the filter, or to warn the user that the localization is unusable.

6. Simulation results

The proposed method is evaluated in a simulated environment using ROS (Robot Operating System, www.ros.org) where three robots navigate in a map composed of four landmarks. The chosen trajectories in Fig. 4 allow for multiple collaboration situations. The contribution of the collaboration for localization and map update is studied with an emphasis on the FDIR step.

The robots are equipped with odometry measurements and simulated lidar with a detection range of 6 m. The landmarks are placed on the corners of a 10 m side square. The initial map contains incorrectly positioned landmarks, with an average vector error of $\vec{0}$. The initial covariances on the landmarks are well-chosen to represent the errors. Initially, all robots have the same map.

6.1. Scenario

Fig. 5 illustrates the collaboration instants between the robots. For instance, robot 1 and 3 collaborate from $t = 10$ s to $t = 15.5$ s via landmark 1, and from $t = 19$ s to $t = 34$ s via landmark 2. A collaboration involving all three robots is observed between $t = 36$ s and $t = 39.5$ s through landmark 4.

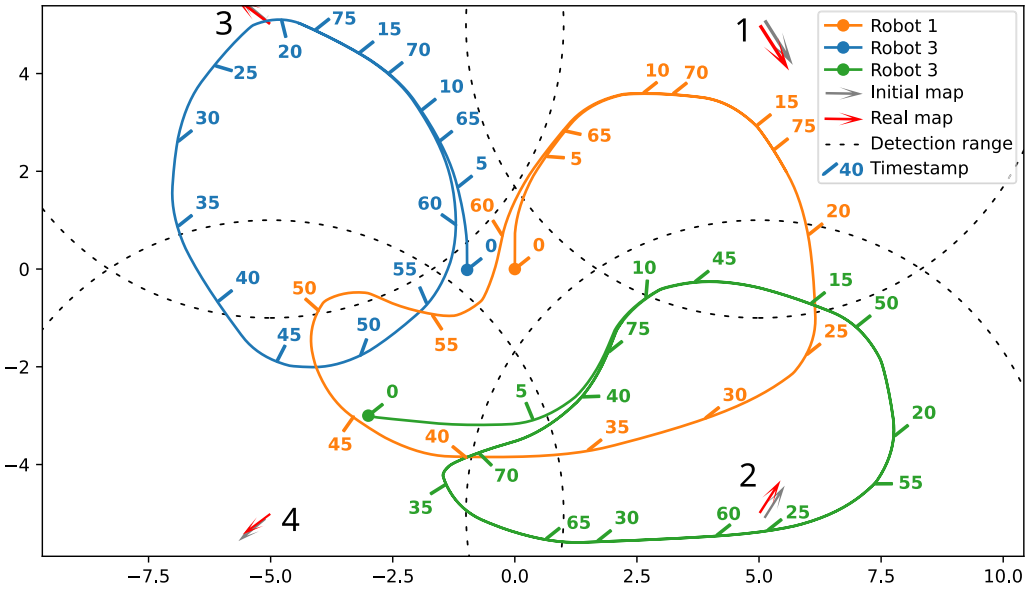


Fig. 4. Trajectories of the three robots with four landmarks drawn with arrows. The dotted circles represent the detection zone where a landmark could be detected by a robot. The trajectory of each robot is shown in different color with the timestamps in seconds.

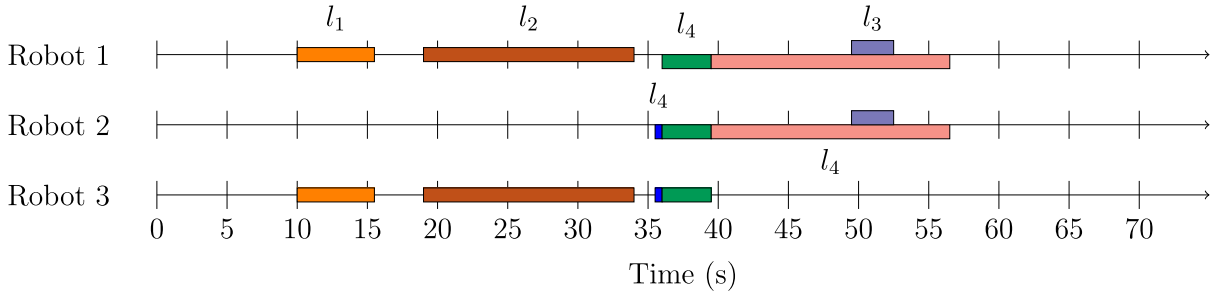


Fig. 5. Collaboration between robots. When two robots collaborate through a landmark, a rectangle of the same color is drawn at the corresponding timestamps, and the landmark observed during the collaboration is indicated.

6.2. Impact of collaboration on localization and map estimation

In this section, the presented results are averaged over 100 runs, where each run has a different noise realization drawn from the same statistical distribution. Therefore, the presented results and metrics capture the overall performance of the method rather than focusing on a single noise realization.

Regarding the choice of Q_p, we have studied its effect on the results. Theoretically, a value of Q_p close to zero can lead to inconsistency, and therefore to a degradation in KLA performance. This occurs when a collaborating robot has an estimate of the ego-robot's pose with an incorrect small covariance. In such cases, the KLA will lead poor result. In our tests with different Q_p values, we observed that the errors did not show significant differences between the values of Q_p, when the latter is different from $\mathbb{0}$. When Q_p is large, there is no significant loss thanks to the KLA. In practice, we used:

$$Q_p = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.5 \end{pmatrix}. \tag{41}$$

Figs. 6(a) and 6(b) show the errors with the associated uncertainty values for robots 1 and 2. Robot 3's results are similar and have been omitted for simplification. The collaborative and standalone methods are compared.

The difference between the collaborative and the standalone methods is obvious. At the end of the simulation, robot 2 shows a better localization and a reduced uncertainty thanks to the collaboration.

Table 4
Robots and Landmarks mean Euclidean errors (cm) and final errors.

Robots	1	2	3		All	All Final error
Collaboration	**4.10**	**9.84**	**5.21**		**6.39**	**1.75**
Standalone	7.58	14.12	7.17		9.62	4.89
Landmarks	**1**	**2**	**3**	**4**	All	All Final error
Collaboration	**4.89**	**5.06**	**7.33**	**7.22**	**6.12**	**1.03**
Standalone	9.00	9.49	11.68	12.02	10.54	7.99

Robot 1 shows similar final result with both methods, which can be attributed to its observation of all the landmarks.

Table 4 also confirms that collaboration greatly improves the localization of all robots.

Some results on map update are shown in Fig. 7. The estimation of landmark 2 by robot 1 (Fig. 7(a)) is improved in standalone and collaborative modes. However, the collaborative method allows for faster update. At the end of the trajectory, the errors in standalone and collaborative modes are the same since the robot observes all landmarks during its trajectory, so the additional information from collaboration is limited. Fig. 7(b) shows the positioning error of landmark 2 by robot 2. This robot does not directly observe the landmark. Therefore, in standalone mode, this landmark cannot be updated. However, in the collaborative case, it can be corrected through collaboration with robot 3. This improvement will benefit robot 2 in the future when it

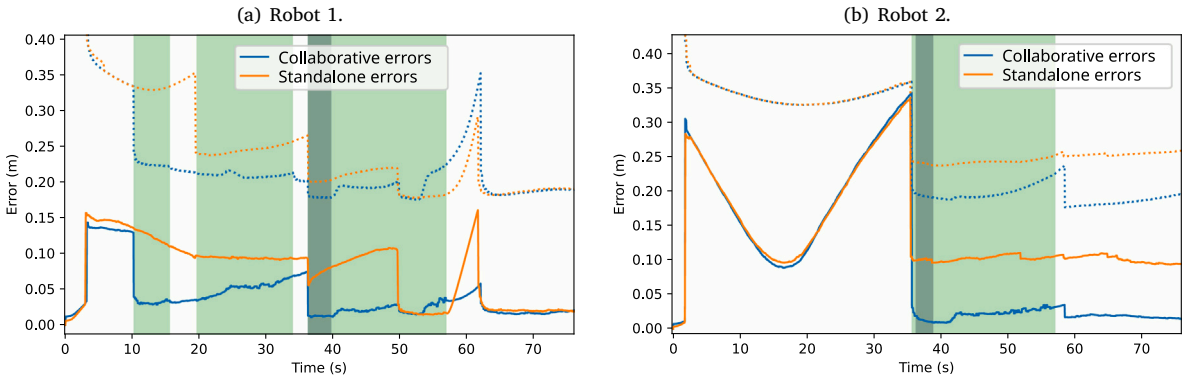


Fig. 6. Euclidean position errors of two robots (in solid lines), averaged over 100 runs, with the associated uncertainty bounds at 99.97 % (Rayleigh distribution) (in dots). Green areas represent the collaboration periods: light green for two robots, dark green for three.

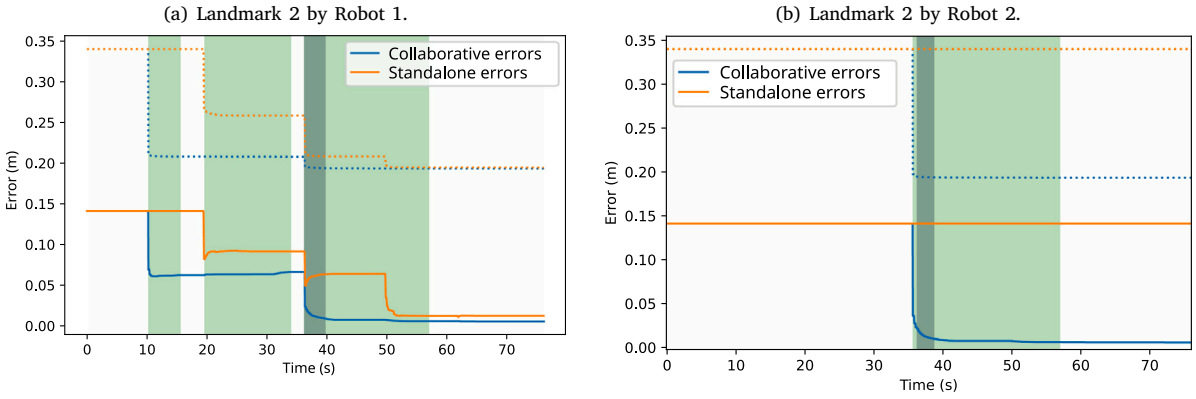


Fig. 7. Euclidean position error of landmark 2 as estimated by (a) robot 1 and (b) robot 2, averaged over 100 runs. The dotted line represents the associated uncertainty bound at 99.97 %.

observes this landmark, as it will already have a more accurate estimate with reduced uncertainty.

Table 4 shows that the mean error of the map decreases from 10.54 cm in standalone mode to 6.12 cm in collaborative mode. The final error decreases from 7.99 cm in standalone mode to 1.03 cm in collaborative mode. These values give an idea about map convergence. Collaboration therefore plays an essential role in correcting the map on a global scale. Since the robots do not observe all the landmarks in standalone mode, their map cannot converge to a small error.

Regarding the uncertainty, all methods correctly compute the uncertainty domains. We can note that the covariances on the position of robots and landmarks are overestimated, as 100 % errors are inside the uncertainty regions associated with a confidence of 99.97 % (4σ of a Rayleigh distribution, as errors are Euclidean distances). Moreover, we can observe a reduction in uncertainty for both robots and landmarks in the collaborative case, while maintaining consistency, which shows another benefit of collaboration.

In addition to the comparison with a standalone method, the proposed decentralized architecture based on SKF was also evaluated against a centralized EKF-based architecture, as reported in our previous work [49]. The decentralized architecture offers advantages such as reduced computation time (see Section 4.3) and the elimination of a single point of failure. However, these benefits come at the expense of some loss in accuracy (about 30 %), which is expected since the centralized architecture is optimal and takes advantages of all available information. It should be noted that the decentralized architecture based on SKF maintains estimation consistency in a manner comparable to the centralized architecture.

6.3. Fault detection, isolation and recovery performance

This section focuses on FDIR evaluation, including comparisons with methods that do not account for faults and with a simple FDE. The importance of map fault detection is highlighted.

6.3.1. Evaluation with faulty observations

Observation faults are injected randomly between 10 and 15 s and between 20 and 40 s for the three robots, with an occurrence probability of 30 %. The faults are injected with an average bias of 20 cm, verifying that no more than two faults occur simultaneously.

When the detection residual of a given robot exceeds the threshold, the isolation residuals are computed to determine the origin of the fault. The detection threshold is determined from a χ^2_n where n is the degree of freedom chosen from the rank of the observation matrix H, and with a false alarm probability of 0.002. From our tests, using rank(H) for the degree of freedom leads to the best results. This fault detection procedure is decentralized and is carried out at the level of each robot independently of the others.

Each robot individually computes its observation residuals. In parallel, when robots collaborate, each computes the residuals of the observations received and landmarks. A binary vector of residuals is determined after a thresholding step, using a χ^2_3 distribution with 3 degrees of freedom with a false alarm probability of 0.002. This vector is then compared to the signature matrix to determine the origin of the fault. In this case, only observation faults are isolated and excluded. On the 152 faults injected into the observations of the three robots, only one was not detected and excluded, corresponding to a 0.66 % miss-detection rate.

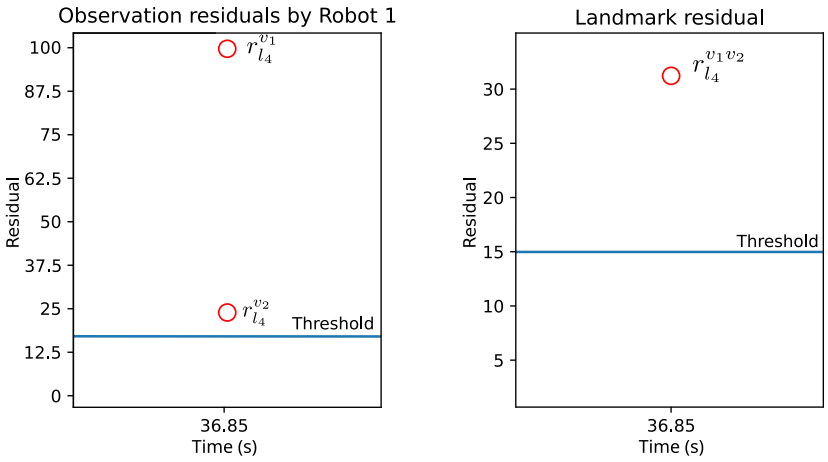


Fig. 8. Residuals (small circles) as computed by robot 1 when two observations of a same landmark are faulty simultaneously. Left figure: observation residuals as computed by robot 1 ($r_{l_4}^{v_1}$ and $r_{l_4}^{v_2}$), right figure: landmark residual ($r_{l_4}^{v_1,v_2}$). The thresholds are shown in blue. All residuals exceed their threshold here.

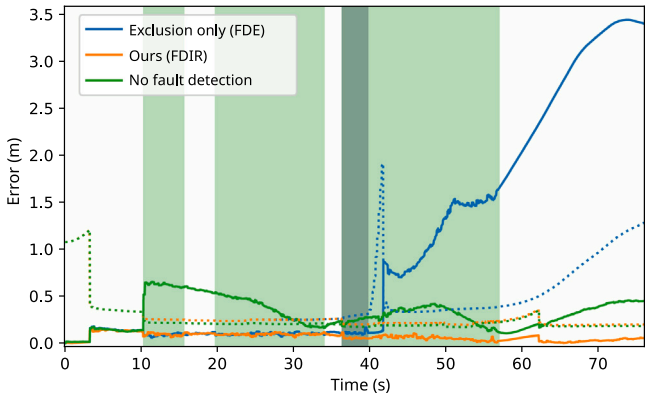


Fig. 9. Position Euclidean errors (in solid lines) of robot 1 with FDIR (orange), with FDE only (blue), and without diagnosis (green), with the associated uncertainty bounds (in dots). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 5
Mean Euclidean errors of robots and landmarks (cm) with observation and landmark faults (single run).

	FDIR	FDE	No Diagnosis
Robot 1	**8.83**	103.26	39.39
Robot 2	**11.78**	97.47	32.19
Robot 3	**11.10**	15.52	53.81
All robots	**10.57**	72.09	41.79
Landmark 1	11.33	**11.28**	64.43
Landmark 2	**13.26**	14.64	50.38
Landmark 3	**11.94**	13.56	43.58
Landmark 4	**23.72**	38.41	72.82
All landmarks	**15.06**	19.47	57.81

The simulation was also designed with injected simultaneous faults. The case of two faulty observations of the same landmark 4 by two different robots appears at $t = 36.85$ s and is presented in Fig. 8. Two robots 1 and 2 collaborate through landmark 4. The two observation residuals of landmark 4 by the two robots exceed their threshold, and the residual of landmark 4 also exceeds its threshold. Comparing with the signature matrix, it corresponds to the case where both observations are faulty, leading to their exclusion from the data fusion. If the fault had originated from the landmark and not from both observations, the landmark residual would have been below its threshold. In that case, the landmark would have been corrected in the map.

6.3.2. Evaluation with faulty landmarks

A bias of 0.3 m is added to landmark 4 in the x and y directions. The initial variance of this landmark is 0.01 m^2 which does not represent this bias. An angle error of 0.25 rad is also added.

Fig. 9 shows the errors of robot 1 with different methods (without diagnosis, with a simple FDE and with our proposed approach based on FDIR). The FDIR allows for better accuracy while maintaining consistency. From $t = 35.5$ s, robots 2 and 3 collaborate by observing landmark 4, which leads to the correction of the landmark faults (Fig. 10(a)). A few moments later, at $t = 36$ s, robot 1 collaborates with the other two, which have already corrected their map. The KLA helps this robot to immediately correct its map without detecting a fault (Fig. 10(b)). Note that the map cannot be corrected before the collaboration between robots.

The method based simply on the exclusion of observations (FDE) leads to a drift for robot 1 (Fig. 9). Indeed, at t = 36 s, robot 1 observes the erroneous landmark, and it collaborates with the other two robots, which observe the same landmark. The observations of the erroneous landmark are then excluded. As robot 1 only observes this landmark, it continues with its odometry, which leads to an increase in its covariance. Thus, the value of the observation residual decreases, until the observations are no longer excluded, causing the divergence. Regarding robot 3, it does not diverge. Therefore, the observations of the erroneous landmark are always excluded and thus this landmark is never corrected (Fig. 10(a)).

6.3.3. Combination of faults

Observation and landmark faults are now combined. Table 5 shows that the FDIR method corrects greatly the map and the localization of the robots. However, landmark errors are large on average, which is due to the convergence time. At the end of the scenario, the errors are much lower. It can be seen that the classical FDE strategy may struggle in some cases and can lead to divergence. Indeed, FDE simply excludes the observations associated with a faulty landmark, whereas our proposed approach can identify a landmark fault and correct it, thanks to the collaboration.

These results validate the proposed approach prior to its evaluation with real data.

7. Evaluation with real data

The method is applied to the localization of vehicles equipped with 3D lidar sensors in urban environments with a prior map obtained

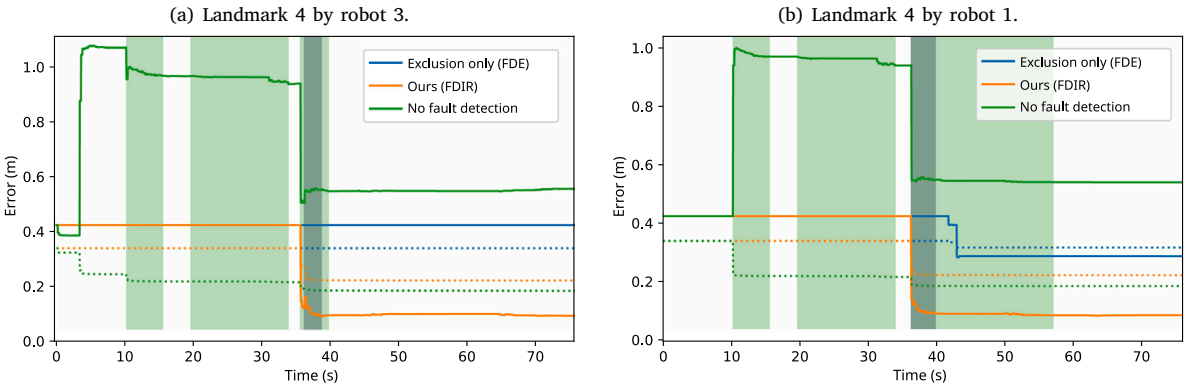


Fig. 10. Position Euclidean errors (in solid lines) of landmark 4 with FDIR, with FDE only, and without diagnosis, with the associated uncertainty bounds (in dots).

from OSM. In these environments, GNSS availability and accuracy are degraded, failing to meet the requirements for autonomous navigation. Buildings have the advantage of being stable over the seasons and undergoing minimal changes over time. Additionally, it is easy to obtain a map of the building facades using OSM.

This section presents the observation generation method and the results obtained on real data. The performance is studied with the proposed FDIR architecture.

7.1. Observation generation

Building facades observations are extracted from lidar point clouds. These observations are relative poses in the vehicle frame and correspond to the middle of the building facades in 2D. Therefore, the observation of facade l_j by vehicle v_i follows the observation model of Eq. (12).

The map from OSM consists of polylines. Landmark poses are then defined from the midpoints of the segments and their normals.

The generation of the observations from the lidar point cloud follows five steps that are detailed in [50]:

1. **Classification**: The buildings points are extracted using Cylinder3D [51].
2. **Registration**: The Normal Distributions Transform (NDT) is used to align the points classified as buildings with the map [52]. Each facade has a vertical grid representation, with cells filled with Normal Distributions (Fig. 11) [53].
3. **Clustering**: Each point of the aligned point cloud is associated to the closest facade within a maximum distance of 1 m.
4. **RANSAC**: RANSAC is applied [54] to remove final outliers.
5. **Observation generation**: An observation in the form of a pose is constructed using the most apart points E_1 and E_2 obtained from the projected inliers to the RANSAC model. The pose is defined as the middle of the segment $[E_1 E_2]$, with orientation obtained from the normal.

7.2. Data acquisition and scenario

A data acquisition was conducted with three experimental vehicles (Fig. 11), each equipped with a 32-layers Velodyne lidar, an odometry sensor, a post-processed RTK IMU SPAN-CPT for ground-truth, and a GNSS receiver for initialization only. The experiment was carried out on the campus of the Université de Technologie de Compiègne, on 90-seconds trajectories in order to create collaboration scenarios (Fig. 12). All vehicles went through a difficult area, constituted of walkway, metallic stairs and buildings (see Fig. 11).

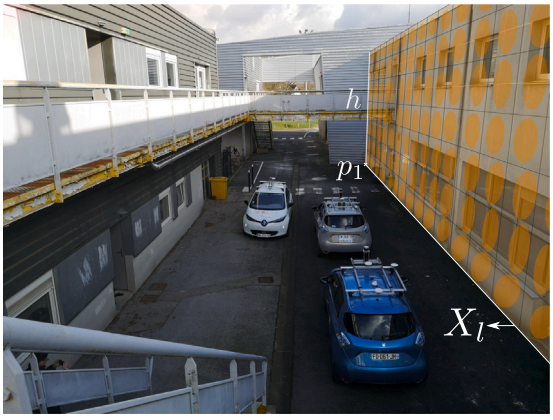


Fig. 11. The three used experimental vehicles. Normal distributions vertical grid is shown on the right, with the middle point l used as a landmark pose.

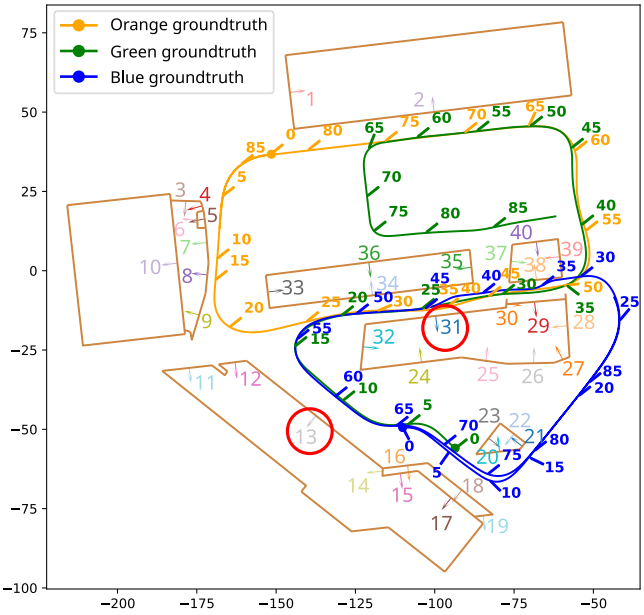
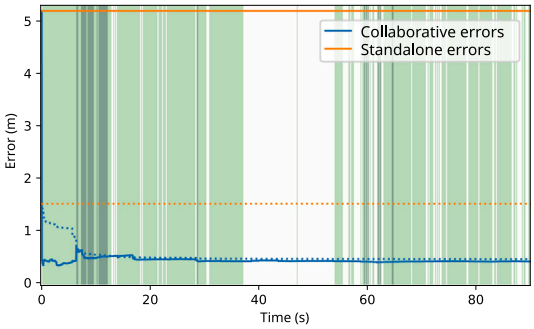


Fig. 12. Trajectories with timestamps and the building map. The numbered landmarks, located at the midpoints of the segments, are the ones that were observed at least once.

(a) Estimation of landmark 13 by Green.



(b) Estimation of landmark 31 by Orange.

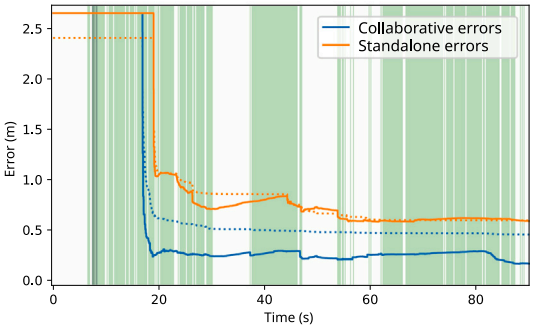


Fig. 13. Position errors of faulty landmarks (in solid lines) and uncertainty bounds (in dots).

Table 6
Mean errors (cm) and consistency (proportions of the errors which are under the 99.7 % uncertainty bound) for the three vehicles.

Methods \Vehicles		Blue	Green	Orange	All
Localization	Collaborative	**39.4**	**50.9**	**57.9**	**49.4**
	Standalone	46.5	62.6	64.6	57.9
Consistency	Collaborative	**99.1**	**95.4**	**82.0**	**92.2**
	Standalone	97.7	85.3	77.7	86.9

The names of the vehicles correspond to the color of their trajectories in Fig. 12. At the beginning, Blue and Green collaborate until $t = 12$ s, passing each others. Orange is in standalone until $t = 8$ s, where it joins the other vehicles and collaborate through landmark 13. Between $t = 12$ s and $t = 30$ s, Orange and green collaborate in the difficult area. Around $t = 30$ s, Blue collaborates with Green then Orange for around twenty seconds. Orange follows Green and continue to collaborate, through the landmarks 2 and 36 (among others) until $t = 65$ s. They can also collaborate with Blue through the landmark 9 around $t = 60$ s, which shows that long-range collaboration is possible without direct line of sight.

7.3. Collaborative localization and map update results

In this section, localization and map update results are briefly presented by evaluating the added value of collaboration. The FDIR was enabled to exclude poor quality observations which can occur frequently.

Table 6 presents the mean Euclidean errors of the three vehicles. Collaboration provides an average localization improvement of around 15 % compared to the standalone mode with an average accuracy of 50 cm. This modest improvement can be attributed to the large number of standalone observations (up to nine observations), compared to collaboration, which provides only 1 to 5 additional observations.

Landmark errors remain within the same order of magnitude for both collaborative and standalone methods, with a mean error of 36.0 cm for collaboration compared to 36.7 cm for standalone.

Regarding the uncertainty, Table 6 gives consistency results, representing the proportion of errors exceeding the 99.7 % uncertainty region in the x and y directions, corresponding to 3σ for a normal distribution. Although, if only Blue almost reaches the theoretical proportion of 99.7 %, the collaboration successfully improves the consistency of the estimated vehicle positions. This is attributed to the greater number of observations available through collaboration, allowing a better estimation of both vehicle localization and map.

Table 7
Localization mean errors (m), localization final errors (m) and consistency (proportions of the errors which are under the 99.7 % uncertainty bound) for the three vehicles with and without FDIR, when faults are injected.

Methods \Vehicles		Blue	Green	Orange	All
Loc. mean errors	FDIR	**0.43**	**0.60**	**0.70**	**0.57**
	No Fault detection	0.61	6.21	5.81	4.21
Loc. final errors	FDIR	**0.72**	**1.01**	**0.18**	**0.64**
	No Fault detection	0.87	13.3	8.07	7.40
Consistency	FDIR	**98.7**	**94.3**	**75.3**	**89.4**
	No Fault detection	89.0	38.8	23.0	50.3

7.4. FDIR with simulated faults

To evaluate the added value of the proposed FDIR architecture, biases were injected on observations between $t = 2$ s and $t = 15$ s for all vehicles with a mean value of 1 m (1% occurrence probability) with no more than two faults at a time. Additionally, landmarks 13 and 31 (Fig. 12) were modified to simulate an initial positioning bias of 5 m and 2.6 m respectively, with incorrect initial variance.

Regarding landmarks correction, the faulty landmark 13 is observed from the beginning by Green and Blue and is corrected collaboratively using the FDIR (Fig. 13(a)). In standalone mode, observations of landmark 13 are simply excluded and the landmark cannot be corrected.

The estimation of the erroneous landmark 31 is shown in Fig. 13(b) as estimated by Orange. In collaborative mode, the landmark is directly corrected by the vehicle without going through the FDIR step. Indeed, the injected bias was not significant enough to be detected as a fault. In standalone, the vehicle also manages to naturally correct the landmark. However, the collaborative correction is better, leading to consistent estimation with better accuracy, which is more valuable for localization.

These corrections allow a better estimate of the map, where the average error goes from 49.3 cm in the standalone case to 38.6 cm in the collaborative case. This value is close to that of the case without fault injection (Section 7.3).

Fig. 14 shows the residual used for fault detection as computed by Orange. Orange dots represent the threshold, determined in a similar manner as in simulation using Chi-squared distribution with a degree of freedom obtained from rank of H. The false alarm probability has been manually adjusted based on empirical data analysis. Indeed, with real data, particularly with noisy lidar observations, observation noises do not truly follow a zero-mean Gaussian assumption. Observation faults are detected along the trajectory. Indeed, a significant number of observations present issues due to the observation extraction process.

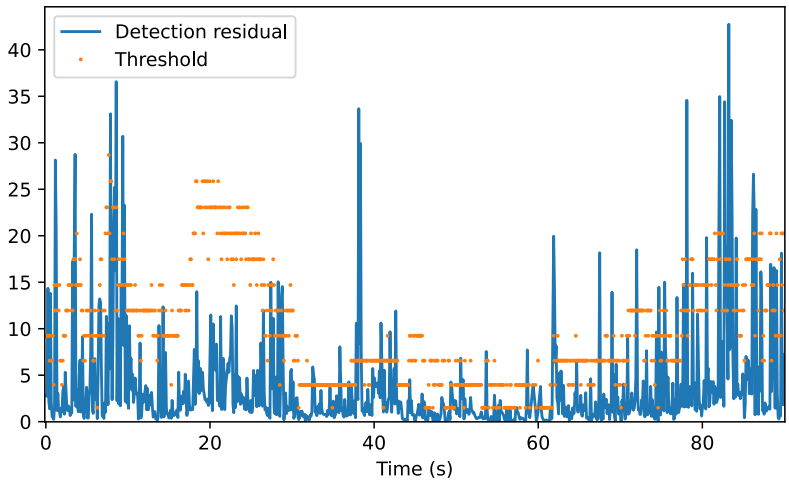


Fig. 14. Detection residual of Orange vehicle (in blue), with the associated threshold (in orange). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

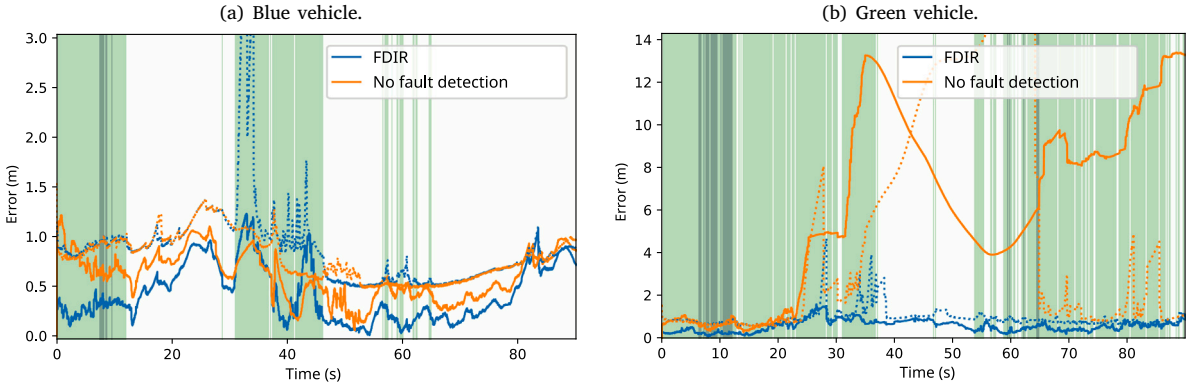


Fig. 15. Vehicles localization errors, with their uncertainty at 99.97 % (dotted line), with and without FDIR when faults are injected. The Orange curves, comparable to those of Green, are omitted. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Fig. 15 presents a comparison of localization errors for Blue and Green, with and without FDIR. The Orange curves, being comparable to those of Green, are omitted. At the start of the trajectory, the localization improvement brought by the FDIR is observable. This effect is not limited to the injected faults but also extends to the difficult area around $t = 40$s, where a drift is observed for Green (and Orange) in the absence of FDIR. Table 7 shows the mean and final errors, as well as the consistency, with and without FDIR. The impact of the FDIR is decisive for Green and Orange vehicles, preventing divergence.

Regarding the consistency, the absence of FDIR leads to inconsistent estimation. The percentage of vehicle localization errors within the 99.7 % uncertainty bound increases from 50.3 % without FDIR to 89.4 % with FDIR (Table 7). Major importance must be then given to the quality of information exchanged within a collaborative system to avoid error propagation between vehicles, which can affect accuracy and consistency.

8. Discussion and comparative analysis

This section provides a comparative analysis of the proposed method against the state of the art, with a focus on approaches using decentralized architectures. Table 8 gives a brief overview of some methods and their characteristics, based on established criteria. Our method addresses several aspects that are typically not considered simultaneously. In particular, the use of a prior map enabling drift-free global localization without GNSS support is rarely explored in the literature. Although the detection of observation faults is considered in some works, the detection and correction of map faults are generally not addressed. Our simulation results demonstrate that this step is essential in scenarios where traditional fault exclusion may lead to divergence.

One of our main objectives is to ensure the integrity of the estimation. To this end, consistent uncertainty bounds must be computed, as we have done in our simulations and experiments. In contrast, such studies are not often conducted with state-of-the-art methods. Furthermore, obtaining uncertainty information is not always straightforward for each method.

Even if some methods (e.g. iMESA [29]) could potentially be modified to address our criteria (map faults, observation faults), significant modeling and adaptation efforts are still required, and proper tuning remains challenging. Regarding data exchanges, each approach shares different types of information, making comparisons in terms of computational and bandwidth usage less relevant.

Therefore, providing a fair analytical comparison is very complex. Many methods lack publicly available codes or require different sensors (e.g. camera). Adapting existing methods to our assumptions is technically non-trivial.

Some results on bandwidth usage of our method are presented below, based on the study in Section 4.3 and Appendix. Considering 32-bit floats and 10 Hz lidar scans, the required bandwidth usage of our proposed method is 172 kB/s in simulation and 6095 kB/s on real-world data. Exchanging the map at 1 Hz instead of 10 Hz would reduce

Table 8
Comparison of different decentralized collaborative methods. Mixed type refers to methods that use filtering and optimization.

		Type	Map (built/update)	Prior map	Map faults detection and correction	Observation faults management	Uncertainty estimation	Real-world experimentation	Data exchanged
SKF C-SLAM with FDIR (Proposed method)		Filter	X	X	X	X	X	Outdoor	Landmark observations, states and landmark map
CL	Luft et al. [25]	Filter					X	Indoor	Relative measurements and state
	Omni-Swarm [26]	Optim.	X			X		Both	Feature keyframes
	Swarm-LIO2 [27]	Mixed					Not evaluated	Outdoor	State and inter-measurements
C-SLAM	Thrun et al. [28]	Filter	X				Not evaluated	Outdoor	Landmark map with robot pose
	Cieslewski et al [30]	Optim.	X					Outdoor	Inter-robot observations and map contributions
	Lázaro et al. [14]	Filter	X	X			Not evaluated	Simulation only	Full state (submap)
	Vemprala et al. [6]	Mixed	X			X	Not evaluated	Both	Image features and feature map
	iMESA [29]	Optim.	X					Outdoor	Observations, inter-robot measurements, and states

the bandwidth usage by 87 % in simulation (22.4 kB/s) and 89 % on real-world (661 kB/s). The reduction in the bandwidth usage is mainly due to the map size which is significantly larger in real-world data compared to simulation.

9. Conclusion

We presented a decentralized architecture with communication constraints that uses an SKF for state estimation and the KLA to merge the estimates provided by each robot. Collaboration is carried out via indirect measurements between robots based on observations of common landmarks, which increases the range of collaboration over 100 m in real-world experiments. To ensure integrity and accuracy, the proposed FDIR stage plays an important role. The use of signature matrices enables a powerful isolation up to two simultaneous faults. The contribution of FDIR conjointly with collaboration was highlighted for map fault isolation. Consequently, the correction of the map is straightforward to perform.

Simulations and real experiments demonstrated that the proposed collaborative approach with FDIR step enhances the accuracy and integrity of localization and mapping. They also support that the FDIR effectively mitigates faults in complex scenarios where multiple faults occur simultaneously. An analytical comparison with state-of-the-art methods is also presented, showing that our approach addresses several aspects that are typically not considered simultaneously.

In future work, we will improve and robustify the feature extraction method, with feature choices guided by the ability to add and remove elements from the map. Optimizations regarding the KLA usage will also be explored.

CRediT authorship contribution statement

Maxime Escourrou: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Joelle Al Hage:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Philippe Bonnifait:** Writing – review & editing, Validation, Supervision, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Communication bandwidth and computational load

A.1. Communication bandwidth

The system needs to exchange data between robots.

Let N be the number of robots and M the total number of landmarks. The full state is then of size $3(N+M)$. At time k, a robot makes h local observations and has c neighboring robots.

Our method first sends the ID of the observed landmarks to the neighboring robots, i.e. a vector of h integers is sent to each of the c robots, resulting in a communication payload of $O(hc)$. A peer-to-peer connection is then established with each neighbor that observes a common landmark. In the worst case scenario, this includes all of the c neighbors. The observation (of size $3\times h$), the full state (of size $3\times(N+M)$) and its covariance matrix (of size $(3\times N+3\times M)^2$) are sent to each neighbor. To simplify the reading, the constant 3 is omitted, as it

does not impact the final order of magnitude. The resulting bandwidth usage is then:

$$O(\underbrace{hc}_{\text{ID messages}} + c\underbrace{\left(h + (N+M) + (N+M)^2\right)}_{\text{message size}}), \tag{A.1}$$

$$= O\left(hc + hc + c(N+M) + c(N+M)^2\right). \tag{A.2}$$

Keeping only superior orders:

$$= O\left(c(N+M)^2\right) \tag{A.3}$$

In real scenario, if there is a high number of robots and a large map, c would be a small fraction of the total number of robots (i.e. $c \ll N$). The bandwidth usage is then, at the higher order:

$$O\left(N^2 + M^2\right). \tag{A.4}$$

Some optimizations can be done in order to improve bandwidth performance. For example, if KLA is only computed at strategic moments, the map only needs to be exchanged at those times, which leads to reduced bandwidth usage. In this case, the full state is not exchanged and the bandwidth would be:

$$O(\underbrace{hc}_{\text{ID messages}} + c\underbrace{\left(h + 1 + 1^2\right)}_{\text{message size}}), \tag{A.5}$$

$$= O\left(hc + hc + c\right). \tag{A.6}$$

Keeping only superior orders:

$$= O\left(hc\right). \tag{A.7}$$

Comparison with channel filter

Suppose that the robots have a connected communication graph (i.e. it exists a path between every pair of nodes), and a spanning tree is determined. This represents the most advantageous and general assumption under which the Channel Filter [47] can operate effectively.

At time k, a robot observes h landmarks and sends its updated information vector and information matrix to its neighbors. Each neighbor then propagates its own updated information in the same manner, until reaching the leaves of the tree. This implies that there are $N-1$ exchanges, each involving a vector of size $3 \times (N+M)$ and a matrix of size $(3 \times N + 3 \times M)^2$. The size of the observation does not affect the bandwidth, as the observations are not transmitted. The bandwidth complexity is then:

$$O(\underbrace{(N-1)}_{\substack{\text{number of}\\ \text{messages}}} \underbrace{\left((N+M) + (N+M)^2\right)}_{\text{message size}}), \tag{A.8}$$

$$= O\left(N^3 + 2MN^2 + M^2N - M^2 - MN - N - M\right). \tag{A.9}$$

Keeping only superior orders:

$$= O\left(N^3 + 2MN^2 + M^2N\right). \tag{A.10}$$

Therefore, the proposed method uses less bandwidth than the traditional decentralized fusion algorithm using Channel Filter. Moreover, the proposed method does not rely on permanent communication, and a message loss has no critical impact.

A.2. Computation complexity

In this work, the SKF is not only essential for achieving consistent state estimation without the need for constant communication, but it also offers lower computational complexity compared to the conventional EKF.

First, let us define the different operation complexities:

- The complexity of multiplying two matrices of sizes $n \times k$ and $k \times m$ is $O(nkm)$, using basic algorithm.
- The complexity of the matrix inversion is $O(n^3)$ using Gauss–Jordan elimination.

Prediction step

The prediction step is the same for both our method using the SKF and the EKF.

Update step

The update step is divided into multiple operations.

For the update step, the computation of S, given by Eq. (28), has a complexity of $O(ms^2 + m^2 s)$, where m is the size of the observation vector that contains local and collaborative observations, and s is the size of the s part (which includes the collaborating robots, and the map). The inversion of the S matrix, of size $m \times m$ has a complexity of $O(m^3)$. Then, we can compute the Kalman gain K whose final complexity (counting S^{-1} computation) is:

$$O(\underbrace{2ms^2 + 2sm^2}_{\text{products}} + \underbrace{m^3}_{\text{inversion}}). \tag{A.11}$$

The state update $X_{s,k|k}$, given by Eq. (22), has a multiplication complexity of $O(sm)$.

The covariance update is performed in two steps. The computational complexity of P_{ss}, as defined by Eq. (24), is:

$$O\left(ms^2 + s^3\right), \tag{A.12}$$

while the one of P_{sp}, given by Eq. (25), is:

$$O\left(ms^2 + s^2 p\right). \tag{A.13}$$

where p is the size of the p part (non collaborating robots).

The total complexity of the update step is then:

$$O\left(s^3 + m^3 + 2m^2 s + 4ms^2 + s^2 p + sm\right). \tag{A.14}$$

Considering that the number of observations is small compared to the map size (i.e. $m \ll 3 \times M < s$), the complexity of the SKF is then:

$$O\left(s^3 + s^2 p + s\right). \tag{A.15}$$

The EKF has almost the same equations, like if s includes all the elements and $p = 0$. If we denote n the size of the state vector that contains all the robots in the system and the map ($n = s + p$), the computational complexity of the EKF is:

$$O\left(m^3 + n^3 + 3mn^2 + 2m^2 n + mn\right). \tag{A.16}$$

Considering that the number of observations is small compared to the map size, it leads to:

$$O\left(n^3 + 3n^2 + 3n\right). \tag{A.17}$$

At the higher orders, the complexity of the SKF is then $O\left(s^3 + s^2 p\right)$. By rewriting $p = n - s$, it leads to $O\left(ns^2\right)$. The complexity of the EKF is $O\left(n^3\right)$.

In almost all cases, we have $s < n$. When s is sufficiently smaller than n, the SKF exhibits lower computational complexity compared to the EKF.

KLA

The KLA has a complexity of $O\left(n^3\right)$ due to the inversion of the full state covariance. However, optimization could be done. For example, the KLA can be computed only at strategic moments, such as when two robots start a collaboration or at a low frequency during ongoing collaboration.

Summary

- The communication bandwidth usage scales as $O\left(N^2 + M^2\right)$ where N and M are the number of robots and landmarks, respectively. For a fixed map size, the communication complexity with respect to the number of robots N is also of order 2.

- The computation complexity for SKF scales efficiently with the number of robots since the s part contains only the subset of collaborating robots.
- The use of KLA introduces a complexity of $O\left(n^3\right)$ where n is the full state size. For a fixed map size, the complexity is of order 3 with respect to the number of robots. However, computing the KLA only at strategically chosen times significantly reduces its impact on overall computational load.

Data availability

Data can be shared on request except the map groundtruth (we do not have the right to share)

References

[1] C. Bao, Y. Hu, Z. Yu, Current Study on Multi-Robot Collaborative Vision SLAM, Appl. Comput. Eng. 35 (1) (2024) 80–88.

[2] C. Chen, H. Wu, Y. Ma, J. Lv, L. Li, Y. Liu, LiDAR-Inertial SLAM with Efficiently Extracted Planes, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2023, pp. 1497–1504.

[3] S. Lee, J.-H. Ryu, Autonomous vehicle localization without prior high-definition map, IEEE Trans. Robot. 40 (2024) 2888–2906.

[4] A. Schaefer, J. Vertens D. Büscher, L. Luft, W. Burgard, Long-term vehicle localization in urban environments based on pole landmarks extracted from 3-D lidar scans, Robot. Auton. Syst. 136 (2021) 103709.

[5] P.-Y. Lajoie, B. Ramtoula, F. Wu, G. Beltrame, Towards Collaborative Simultaneous Localization and Mapping: A Survey of the Current Research Landscape, Field Robot. 2 (1) (2022) 971–1000.

[6] S.H. Vemprala, S. Saripalli, Collaborative Localization for Micro Aerial Vehicles, IEEE Access 9 (2021) 63043–63058.

[7] R. Zanetti, C. D'Souza, Recursive implementations of the Schmidt-Kalman 'consider' filter, J. Astronaut. Sci. 60 (3–4) (2013) 672–685.

[8] G. Battistelli, L. Chisci, Kullback–Leibler average, consensus on probability densities, and distributed state estimation with guaranteed stability, Automatica 50 (3) (2014) 707–718.

[9] G. Bresson, Z. Alsayed, L. Yu, S. Glaser, Simultaneous Localization and Mapping: A Survey of Current Trends in Autonomous Driving, IEEE Trans. Intell. Veh. 2 (3) (2017) 194–220.

[10] C. Campos, R. Elvira, J.J.G. Rodríguez, J.M.M. Montiel, J.D. Tardós, ORB-SLAM3: An accurate open-source library for visual, visual–Inertial, and multimap SLAM, IEEE Trans. Robot. 37 (6) (2021) 1874–1890.

[11] E. Mendes, P. Koch, S. Lacroix, Icp-based pose-graph slam, in: IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR), Lausanne, Switzerland, 2016, pp. 195–200.

[12] L. Paz, P. Jensfelt, J. Tardos, J. Neira, EKF SLAM Updates in O(n) with Divide and Conquer SLAM, in: IEEE International Conference on Robotics and Automation, 2007, pp. 1657–1663.

[13] V. Sezer O.V. Altınpınar, A novel indoor localization algorithm based on a modified EKF using virtual dynamic point landmarks for 2D grid maps, Robot. Auton. Syst. 170 (2023).

[14] M.T. Lázaro, L.M. Paz, J.A. Castellanos P. Piniés, Distributed Localization and Submapping for Robot Formations Using a Prior Map, IFAC Proc. Vol. 46 (10) (2013) 138–145.

[15] H. Mu, T. Bailey, P. Thompson, H. Durrant-Whyte, Decentralised Solutions to the Cooperative Multi-Platform Navigation Problem, IEEE Trans. Aerosp. Electron. Syst. 47 (2) (2011) 1433–1449.

[16] T. Bailey, J. Nieto, J. Guivant, M. Stevens, E. Nebot, Consistency of the EKF-SLAM Algorithm, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, Beijing, 2006, pp. 3562–3568.

[17] H. Strasdat, J.M.M. Montiel, A.J. Davison, Real-Time Monocular SLAM: Why Filter? in: IEEE International Conference on Robotics and Automation, 2010, pp. 2657–2664.

[18] S. Thrun, D. Koller, Z. Ghahramani, A.Y. Ng H. Durrant-Whyte, Simultaneous Mapping and Localization with Sparse Extended Information Filters: Theory and Initial Results, in: Algorithmic Foundations of Robotics V, Springer Tracts in Advanced Robotics, Springer, Berlin, Heidelberg, 2004, pp. 363–380.

[19] P. Geneva, J. Maley, G. Huang, An Efficient Schmidt-EKF for 3D Visual-Inertial SLAM, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, Long Beach, CA, USA, 2019, pp. 12097–12107.

[20] A. Barrau, S. Bonnabel, The Invariant Extended Kalman Filter as a Stable Observer, IEEE Trans. Autom. Control 62 (4) (2017) 1797–1812.

[21] Y. Zhang, T. Zhang, S. Huang, Comparison of EKF Based SLAM and Optimization Based SLAM Algorithms, in: 13th IEEE Conference on Industrial Electronics and Applications, ICIEA, Wuhan, 2018, pp. 1308–1313.

[22] O. Bengtsson, A. Baerveldt, Localization in Changing Environments - Estimation of a Covariance Matrix for the IDC Algorithm, in: IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the Next Millennium, Vol. 4, 2001, pp. 1931–1937.

[23] R. Dubois, A. Eudes, J. Moras, V. Fremont, Dense Decentralized Multi-robot SLAM Based on Locally Consistent TSDF Submaps, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, Las Vegas, NV, USA, 2020, pp. 4862–4869.

[24] S. Roumeliotis, G. Bekey, Distributed Multirobot Localization, IEEE Trans. Robot. Autom. 18 (5) (2002) 781–795.

[25] L. Luft, T. Schubert, S. Roumeliotis, W. Burgard, Recursive Decentralized Collaborative Localization for Sparsely Communicating Robots, in: Robotics: Science and Systems, 2016.

[26] H. Xu, Y. Zhang, B. Zhou, L. Wang, X. Yao, G. Meng, S. Shen, Omni-Swarm: A Decentralized Omnidirectional Visual–Inertial–UWB State Estimation System for Aerial Swarms, 38, (6) 2022, pp. 3374–3394,

[27] F. Zhu, Y. Ren, L. Yin, F. Kong, Q. Liu, R. Xue, W. Liu, Y. Cai, G. Lu, H. Li, F. Zhang, Swarm-LIO2: Decentralized Efficient LiDAR-Inertial Odometry for Aerial Swarm Systems, IEEE Trans. Robot. 41 (2025) 960–981.

[28] S. Thrun, Y. Liu, Multi-Robot SLAM with Sparse Extended Information Filters, in: Robotics Research. The Eleventh International Symposium, Springer Tracts in Advanced Robotics, Springer, Berlin, Heidelberg, 2005, pp. 254–266.

[29] D. McGann, M. Kaess, iMESA: Incremental Distributed Optimization for Collaborative Simultaneous Localization and Mapping, in: Robotics: Science and Systems 2024, Robotics: Science and Systems Foundation, 2024.

[30] T. Cieslewski, S. Choudhary, D. Scaramuzza, Data-Efficient Decentralized Visual SLAM, in: IEEE International Conference on Robotics and Automation, ICRA, 2018, pp. 2466–2473.

[31] S.X. Ding, Model-Based Fault Diagnosis Techniques: Design Schemes, Algorithms and Tools, Advances in Industrial Control, Springer, London, 2013.

[32] T. Goelles, B. Schlager, S. Muckenhuber, Fault detection, isolation, identification and recovery (FDIIR) methods for automotive perception sensors including a detailed literature survey for lidar, Sensors 20 (13) (2020) 3662.

[33] J. Ragot, Aberrant Measurements: Detection, Localization, Suppression, Acceptance and Robustness, Measurement 172 (2021) 108872.

[34] E. Khalastchi, M. Kalech, On Fault Detection and Diagnosis in Robotic Systems, ACM Comput. Surv. 51 (1) (2018) 9:1–9:24.

[35] B. Kellalib, N. Achour, V. Coelen, A. Nemra, Towards Simultaneous Localization and Mapping Tolerant to Sensors and Software Faults: Application to Omnidirectional Mobile Robot, Proc. Inst. Mech. Eng. Part I: J. Syst. Control. Eng. 235 (2) (2021) 269–288.

[36] M. Kheirandish, E.A. Yazdi, H. Mohammadi, M. Mohammadi, A fault-tolerant sensor fusion in mobile robots using multiple model Kalman filters, Robot. Auton. Syst. 161 (2023) 104343.

[37] L. Hachemi, M. Guiatni, A. Nemra, Fault Diagnosis and Reconfiguration for Mobile Robot Localization Based on Multi-Sensors Data Fusion, Unmanned Syst. 10 (01) (2022) 69–91.

[38] H. Hamadi, B. Lussier, I. Fantoni, C. Francis, Data Fusion Fault Tolerant Strategy for a Quadrotor UAV under Sensors and Software Faults, ISA Trans. 129 (2022) 520–539.

[39] V. Gandhi O. Graham Miller, A Survey of Modern Exogenous Fault Detection and Diagnosis Methods for Swarm Robotics, J. King Saud Univ., Eng. Sci. 33 (1) (2021) 43–53.

[40] D. Tarapore, J. Timmis, A.L. Christensen, Fault Detection in a Swarm of Physical Robots Based on Behavioral Outlier Detection, IEEE Trans. Robot. 35 (6) (2019) 1516–1522.

[41] D. Azzalini, A. Castellini, M. Luperto, A. Farinelli, F. Amigoni, HMMs for Anomaly Detection in Autonomous Robots, in: Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS'20, International Foundation for Autonomous Agents and Multiagent Systems, 2020, pp. 105–113.

[42] J. Al Hage, M.E. El Najjar, D. Pomorski, Multi-Sensor Fusion Approach with Fault Detection and Exclusion Based on the Kullback–Leibler Divergence: Application on Collaborative Multi-Robot System, Inf. Fusion 37 (2017) 61–76.

[43] Z.E. Mawas, C. Cappelle, M. El Badaoui El Najjar, Decision tree based diagnosis for hybrid model-based/data-driven fault detection and exclusion of a decentralized multi-vehicle cooperative localization system*, IFAC-PapersOnLine 56 (2) (2023) 7740–7745.

[44] R.A. Carrasco, F. Núñez, A. Cipriano, Fault Detection and Isolation in Cooperative Mobile Robots Using Multilayer Architecture and Dynamic Observers, Robotica 29 (4) (2011) 555–562.
[45] C. Tessier, C. Cariou, C. Debain, F. Chausse, R. Chapuis, C. Rousset, A real-time, multi-sensor architecture for fusion of delayed observations: application to vehicle localization, in: IEEE Intelligent Transportation Systems Conference, 2006, pp. 1316–1321.
[46] A. Lima, P. Bonnifait, V. Cherfaoui, J. Al Hage, Data fusion with split covariance intersection for cooperative perception, in: IEEE International Intelligent Transportation Systems Conference (ITSC), 2021, pp. 1112–1118.
[47] S. Grime, H. Durrant-Whyte, Data fusion in decentralized sensor networks, Control Eng. Pract. 2 (5) (1994) 849–863.
[48] R. Smith, M. Self, P. Cheeseman, Estimating Uncertain Spatial Relationships in Robotics, Autonomous Robot Vehicles, Springer, New York, NY, 1990, pp. 167–193.
[49] M. Escourrou, J. Al Hage, P. Bonnifait, Decentralized Collaborative Localization with Map Update using Schmidt-Kalman Filter, in: 25th International Conference on Information Fusion (FUSION), 2022, pp. 1–8.
[50] M. Escourrou, J. Al Hage, P. Bonnifait, Decentralized Collaborative Localization and Map Update with Buildings, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, 2024, pp. 5696–5703.
[51] X. Zhu, H. Zhou, T. Wang, F. Hong, Y. Ma, W. Li, H. Li, D. Lin, Cylindrical and Asymmetrical 3D Convolution Networks for LiDAR Segmentation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2021, pp. 9934–9943.
[52] M. Magnusson, The Three-Dimensional Normal-Distributions Transform: An Efficient Representation for Registration, Surface Analysis, and Loop Detection (Ph.D. thesis), Örebro University, 2009.
[53] E. Javanmardi, Y. Gu, M. Javanmardi, S. Kamijo, Autonomous Vehicle Self-Localization Based on Abstract Map and Multi-Channel LiDAR in Urban Area, IATSS Res. 43 (1) (2019) 1–13.
[54] R. Schnabel, R. Wahl, R. Klein, Efficient RANSAC for Point-Cloud Shape Detection, Comput. Graph. Forum 26 (2) (2007) 214–226.



Maxime Escourrou received an engineering degree in Computer Science, with a specialty in Real Time and Embedded Systems, and a master's degree in Automatic and Robotic Systems in 2020. In 2023, he received his Ph.D. degree in Robotics and Computer Science from the Université de Technologie de Compiègne, in France. Since 2024, he has been an associate professor at the Université de Toulouse, and he has been part of the LAAS-CNRS. His research focuses on robust multi-robot localization and mapping methods for mobile robotics.



Joelle Al Hage received an engineer's degree in Electrical and Electronic Engineering and a master's degree in Reliability, Identification and Diagnosis from the Lebanese University (2013). In 2016, she received her Ph.D. degree in Robotics and Computer Science from University of Lille in France. From 2017 to 2019 she was a CNRS researcher at SIVALab (a joint Renault-UTC-CNRS laboratory). Since 2019 she has been an associate professor in the Computer Science and Engineering Department at UTC and is carrying out research in the Heudiasyc laboratory. Her research is focused on multi-sensor data fusion with fault tolerance, collaborative localization and mapping and the integrity of localization and perception methods for autonomous vehicles.



Philippe Bonnifait received his Ph.D. in Automatic Control and Computer Science from the Ecole Centrale de Nantes in France in 1997. He is currently a professor in the Computer Science and Engineering Department, Université de Technologie de Compiègne, France. Since 1998 he has been part of Heudiasyc UMR 7253, a joint UTC-CNRS research laboratory. His research interests include intelligent vehicles, and high-integrity positioning and map-matching for autonomous navigation in structured outdoor environments.